

Diagrams, screens and code for the report

Use cases, data flow diagrams, activity, sequence, state and class diagrams, the database and API diagrams, the architecture, the planned and actual timelines, the screens, and the main function code, in the order of the ZCAS undergraduate project guide.

AI-Driven Mobile Health Platform for Prostate Cancer Diagnosis (Zambia)

Made on 2 October 2026 · code pictures from commit b3dd3d4 with uncommitted changes

Drawn from the project itself. All data shown is synthetic. Not a medical device.

The diagrams, screens and code pictures for the final-year report, in the order of the chapters in the ZCAS undergraduate project guide. Each figure has a suggested number and caption, what it shows, and its files.

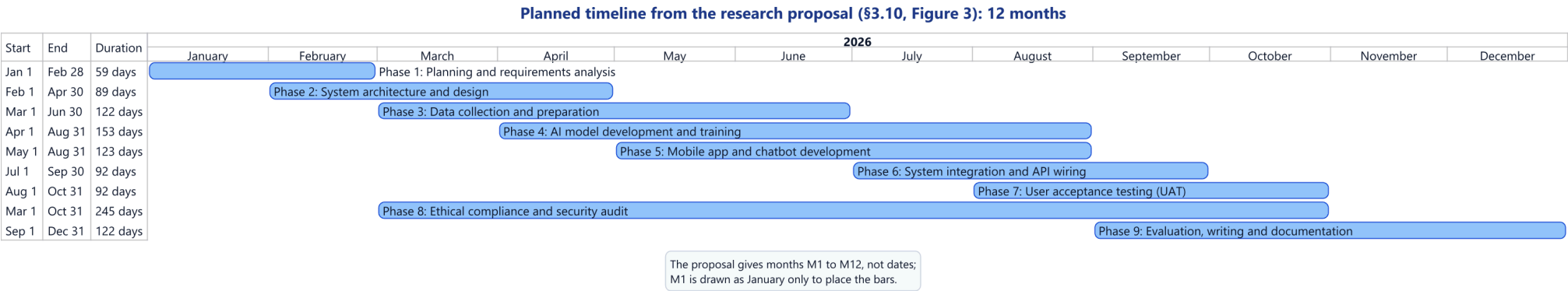
- **To put a figure in Word:** Insert > Pictures > the `.png` file. The PNGs are drawn at high resolution, so they stay sharp when printed.
 - **To restyle a figure in Figma or Canva:** import the `.svg` file. It is a vector drawing, so every box and line can be moved and recoloured.
 - **They are made from the project itself.** The diagrams are text files (PlantUML) in `docs/report/diagrams`. The database diagrams, the API map and the actual timeline are generated from the database schema, the access matrix and the Git history, so they cannot drift from the code. After a change, run `powershell -ExecutionPolicy Bypass -File docs\report\tools\report.ps1 diagrams figures ; report/tools/README.md` explains it.
 - **Honesty notes for the captions:** the AI analysis uses labelled **mock** models (their numbers come from a hash of the job id, not from patient data), all data is **synthetic**, and the knowledge base still awaits a clinician's sign-off. Say so wherever a figure shows them.
 - The numbers below are suggestions; renumber them to fit your report.
-

Chapter 1: Introduction

1.5 Implementation plan

Figure 1.1: Planned timeline (from the research proposal)

The nine phases and twelve months of the proposal's Gantt chart (§3.10, Figure 3), redrawn. The proposal gives months M1 to M12, not dates.



Files: [PNG](#) · [SVG](#) · source [gantt-planned.puml](#)

Figure 1.2: Actual build timeline (from the Git history)

One diamond per commit, on the day it was made, from the first analysis to the latest change. Useful in Chapter 5 to compare the plan with what happened.

What was built, day by day: one diamond per commit (2026-09-23 to 2026-10-01)

Start	End	Duration	September 2026							Oct		
			We	Th	Fr	Sa	Su	Mo	Tu	We	Th	
			23	24	25	26	27	28	29	30	1	
Sep 23	Sep 23	1 day	◆									Phase 0 project analysis, architecture, plan, ADRs (bd5af4b)
Sep 23	Sep 23	1 day	◆									Phase 1 monorepo skeleton, quality gate and CI (6520784)
Sep 23	Sep 23	1 day	◆									Phase 2 schema, migrations, audit chain, storage abstractions (f70d4fe)
Sep 23	Sep 23	1 day	◆									Phase 3 gateway foundation (c32a729)
Sep 23	Sep 23	1 day	◆									Phase 4 authentication, sessions and RBAC (af0dbbb)
Sep 24	Sep 24	1 day		◆								Phase 5 patients, clinical records and consent (d3bf0be)
Sep 24	Sep 24	1 day		◆								Phase 7 Flutter foundation (d726503)
Sep 24	Sep 24	1 day		◆								Phase 6 offline-first sync, Galaxy S9+ device support (5f77bd6)
Sep 24	Sep 24	1 day		◆								Phase 8 patient workflow (7c3aa00)
Sep 24	Sep 24	1 day		◆								Admin page, role permissions, sign-up ID, Zambian colours (c03e70d)
Sep 24	Sep 24	1 day		◆								Separate React admin portal, per-user rate limits, scalability (bfd9cfc)
Sep 24	Sep 24	1 day		◆								Clinical Field Health design system (5bc85f3)
Sep 24	Sep 24	1 day		◆								Phase 10 imaging and histopathology files (e1959f4)
Sep 25	Sep 25	1 day		◆								Phase 11 AI service foundation (f379c26)
Sep 25	Sep 25	1 day		◆								Organise the repository by the six architecture layers (bfb6374)
Sep 25	Sep 25	1 day		◆								Phase 12 explainability pipeline and evaluation endpoint (0fd06c1)
Sep 25	Sep 25	1 day		◆								AI model integration guide and chatbot build plan (beb163b)
Sep 25	Sep 25	1 day		◆								Phase 9 clinician and pathologist workflow (e85f15a)
Sep 28	Sep 28	1 day						◆				Dark mode and settings screen (0aebf4e)
Sep 28	Sep 28	1 day						◆				Phase 14 de-identified FHIR R4 export and SmartCare Pro mock (dae7661)
Sep 28	Sep 28	1 day						◆				Phase 15 security and compliance review (3bbb4ec)
Sep 28	Sep 28	1 day						◆				Activity dashboard for the phone app and a new navigation bar (967c988)
Sep 28	Sep 28	1 day						◆				"modern health app" redesign, all screens, light and dark (12c1d57)
Sep 28	Sep 28	1 day						◆				Phase 16 end-to-end workflows on the live system (763f1a8)
Sep 29	Sep 29	1 day						◆				Phase 17 performance measured against the research targets (a70ecc8)
Sep 29	Sep 29	1 day						◆				Phase 13 milestone 1, knowledge base and offline extractive answers (21b9634)
Sep 29	Sep 29	1 day						◆				Phase 13 milestone 2, chat API with safety rules (2841f7b)
Sep 29	Sep 29	1 day						◆				Phase 13 milestone 3, "Ask a question" and "Ask the assistant" in the app (5f3624f)
Sep 29	Sep 29	1 day						◆				Phase 13 measured and documented (26dcb9b)
Sep 29	Sep 29	1 day						◆				The assistant chats, and Claude can write its answers (7b91756)
Sep 29	Sep 29	1 day						◆				Plain words for chat actions in the dashboard feed (6150f22)
Sep 29	Sep 29	1 day						◆				Awareness blue, modern icons, assistant bot, voice messages and read-aloud (1b32380)
Sep 29	Sep 29	1 day						◆				Use the PC's own PostgreSQL 18 and MongoDB 8.3 (395441a)
Sep 29	Sep 29	1 day						◆				Delete accounts from the admin website (47dcffc)
Sep 29	Sep 29	1 day						◆				New chats, past chats and casual conversation (3fae63c)
Sep 30	Sep 30	1 day						◆				Operations manual: how it was built and how to run it without an assistant (485825f)
Sep 30	Sep 30	1 day						◆				Dev-up.ps1: start when Docker Desktop is not running (bba633c)
Sep 30	Sep 30	1 day						◆				Development log: chat features checked on the emulator; voice recognition s... (9cdccac)
Oct 1	Oct 1	1 day						◆				Operations manual: a tool to rebuild its pictures, PDF and web page (dd1813e)
Oct 1	Oct 1	1 day						◆				Diagrams and figures for the final-year report; proposal gaps listed (0fde88c)
Oct 1	Oct 1	1 day						◆				User acceptance testing kit (proposal §3.8.2, NFR-04) (9104c32)
Oct 1	Oct 1	1 day						◆				Meaning search with MedCPT next to the keyword search (ADR-013) (b3dd3d4)

Files: [PNG](#) · [SVG](#) · source generated from `git log`

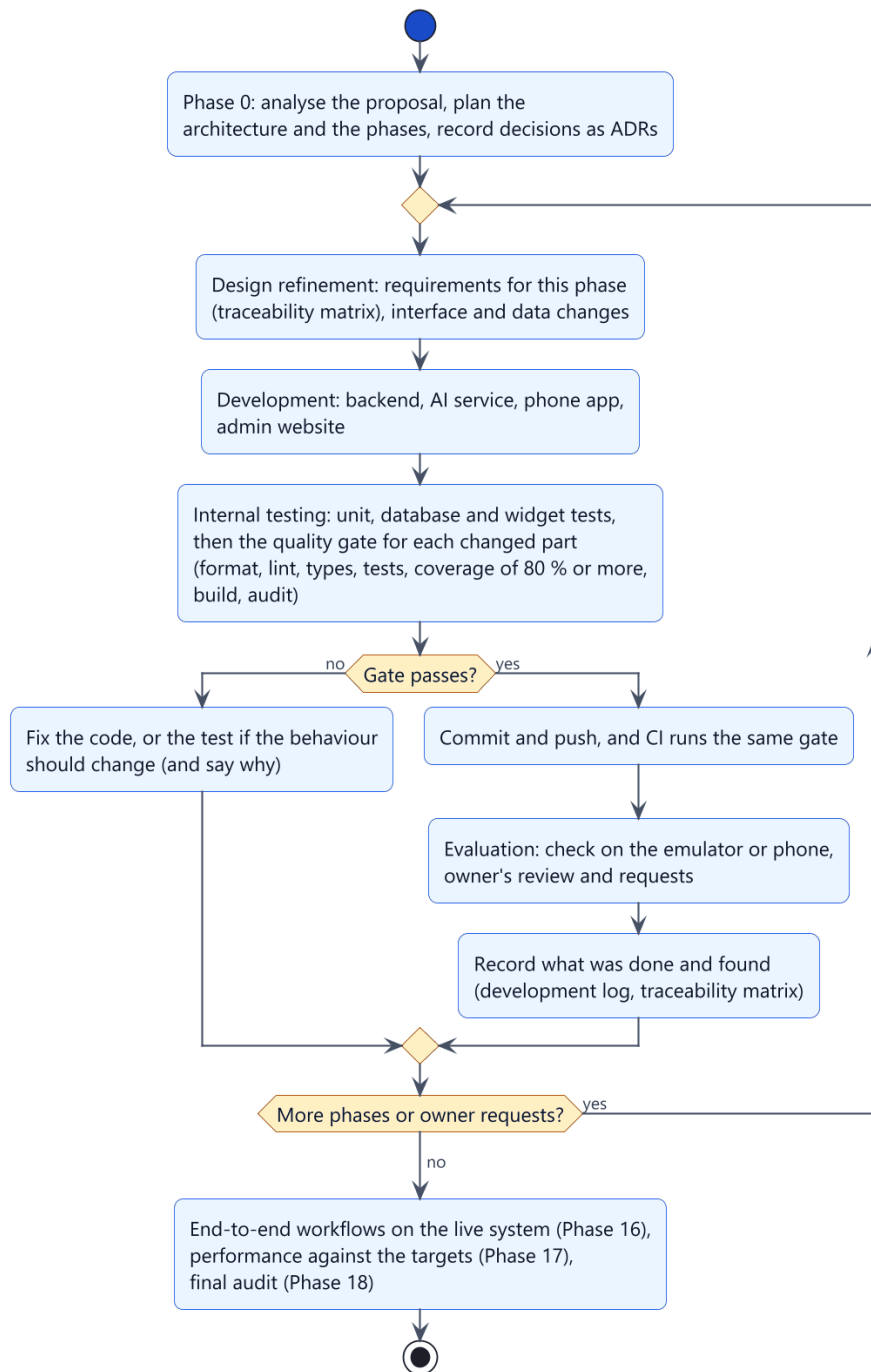
Chapter 3: System analysis and design

3.3 System development method

Figure 3.1: The iterative prototyping cycle as followed

How each phase went through design refinement, development, internal testing (the quality gate) and evaluation, as in proposal §3.2.

How each prototype iteration was built (iterative prototyping, proposal §3.2)

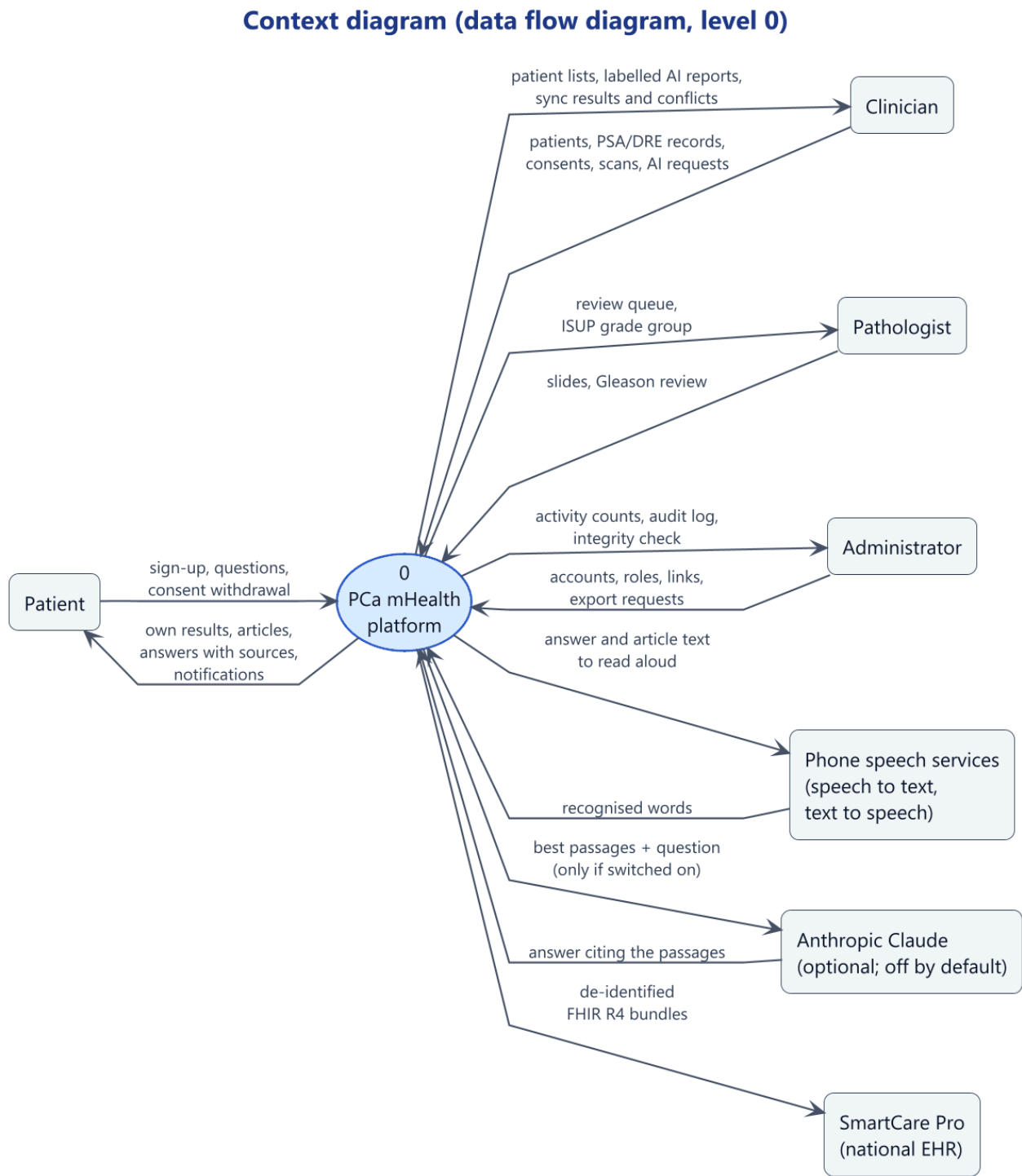


Files: [PNG](#) · [SVG](#) · source [activity-prototyping-cycle.puml](#)

3.4 Data and process modelling

Figure 3.2: Context diagram (DFD level 0)

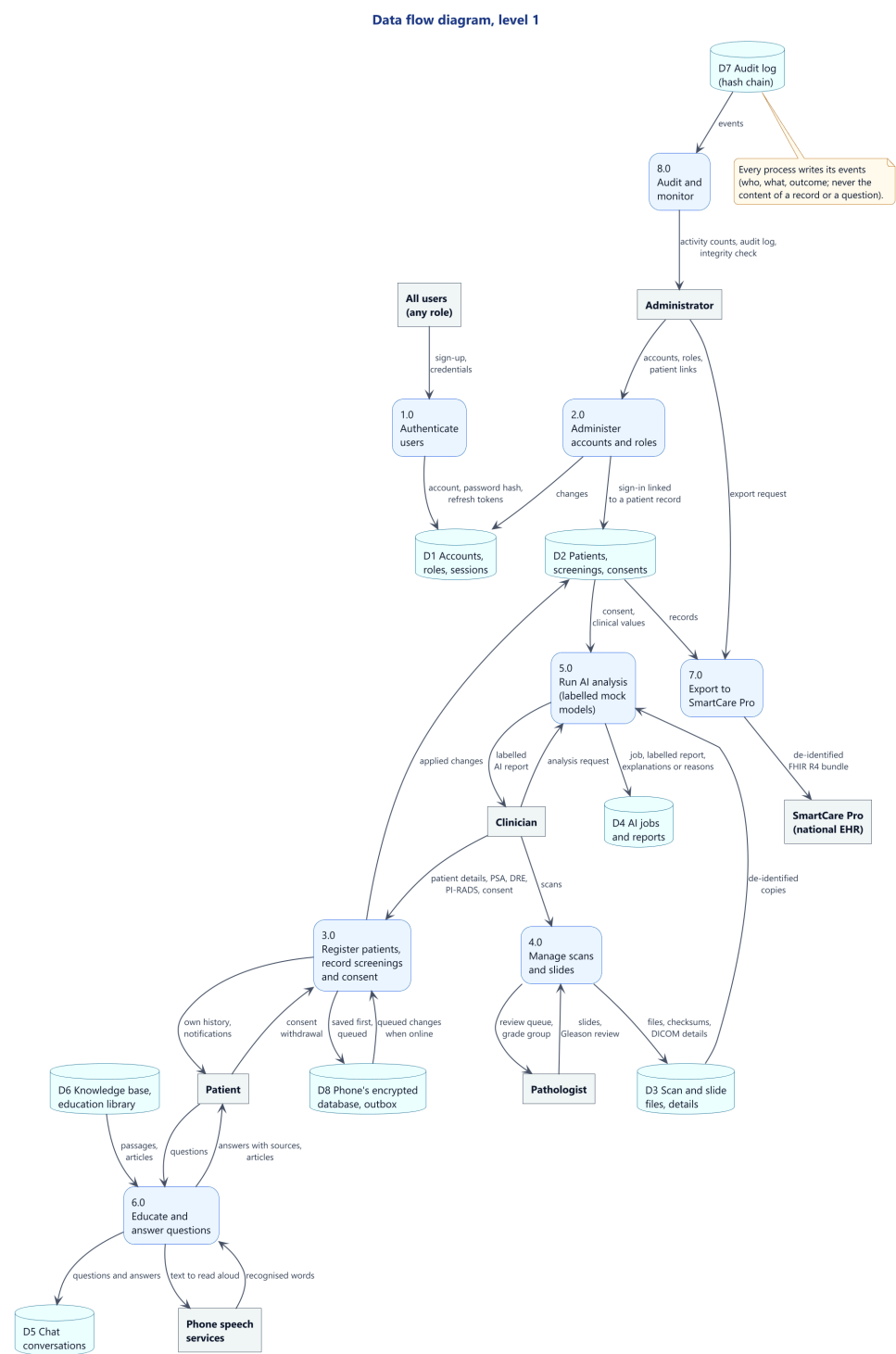
The whole platform as one process, with the people and systems around it and the data that flows between them.



Files: [PNG](#) · [SVG](#) · source [context-diagram.puml](#)

Figure 3.3: Data flow diagram, level 1

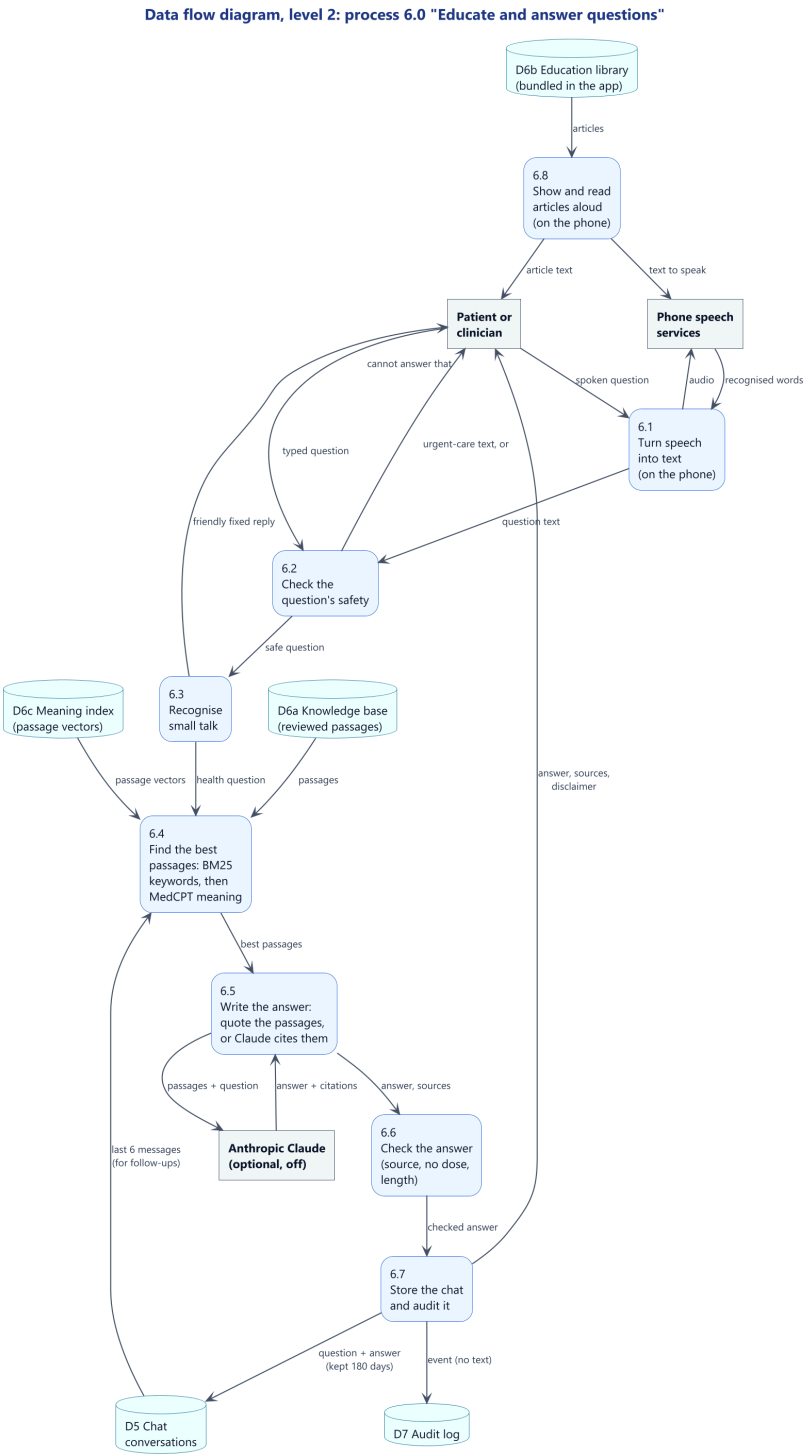
The eight main processes, the eight data stores (D1 to D8) and the flows between them and the users.



Files: PNG · SVG · source dfd-level-1.puml

Figure 3.4: Data flow diagram, level 2: the assistant and education

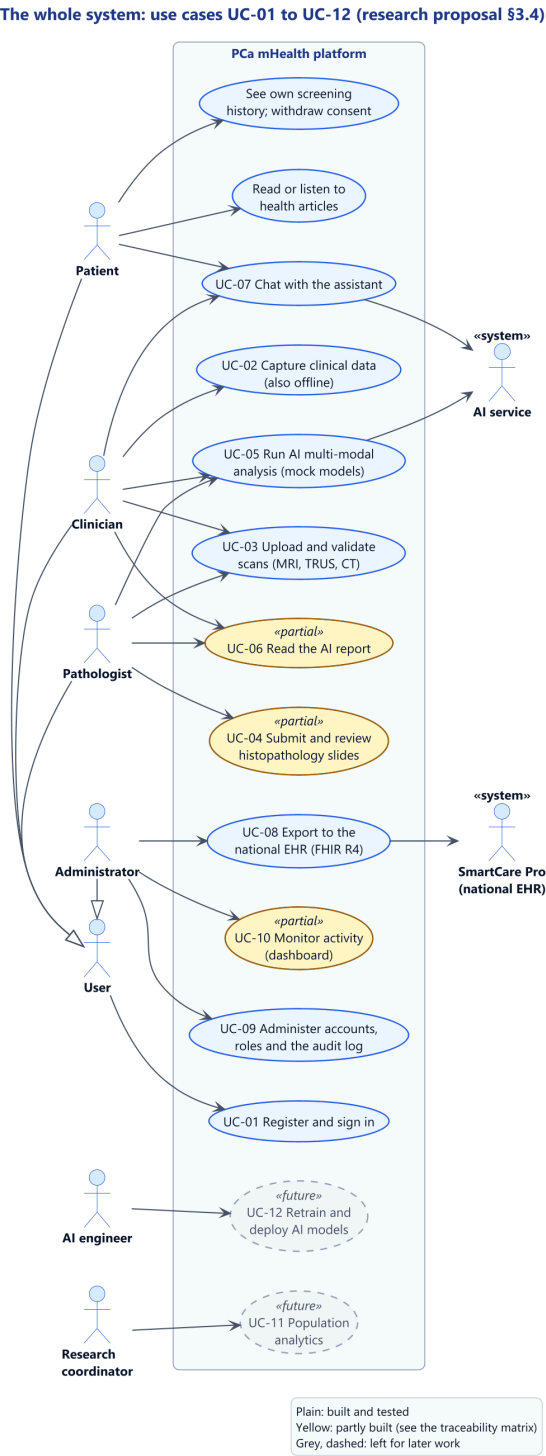
Process 6.0 in detail: speech to text on the phone, the safety check, small talk, the knowledge-base search, writing and checking the answer, storing it, and reading articles aloud.



Files: [PNG](#) · [SVG](#) · source [dfd-level-2-chatbot.puml](#)

Figure 3.5: Use case diagram: the whole system (UC-01 to UC-12)

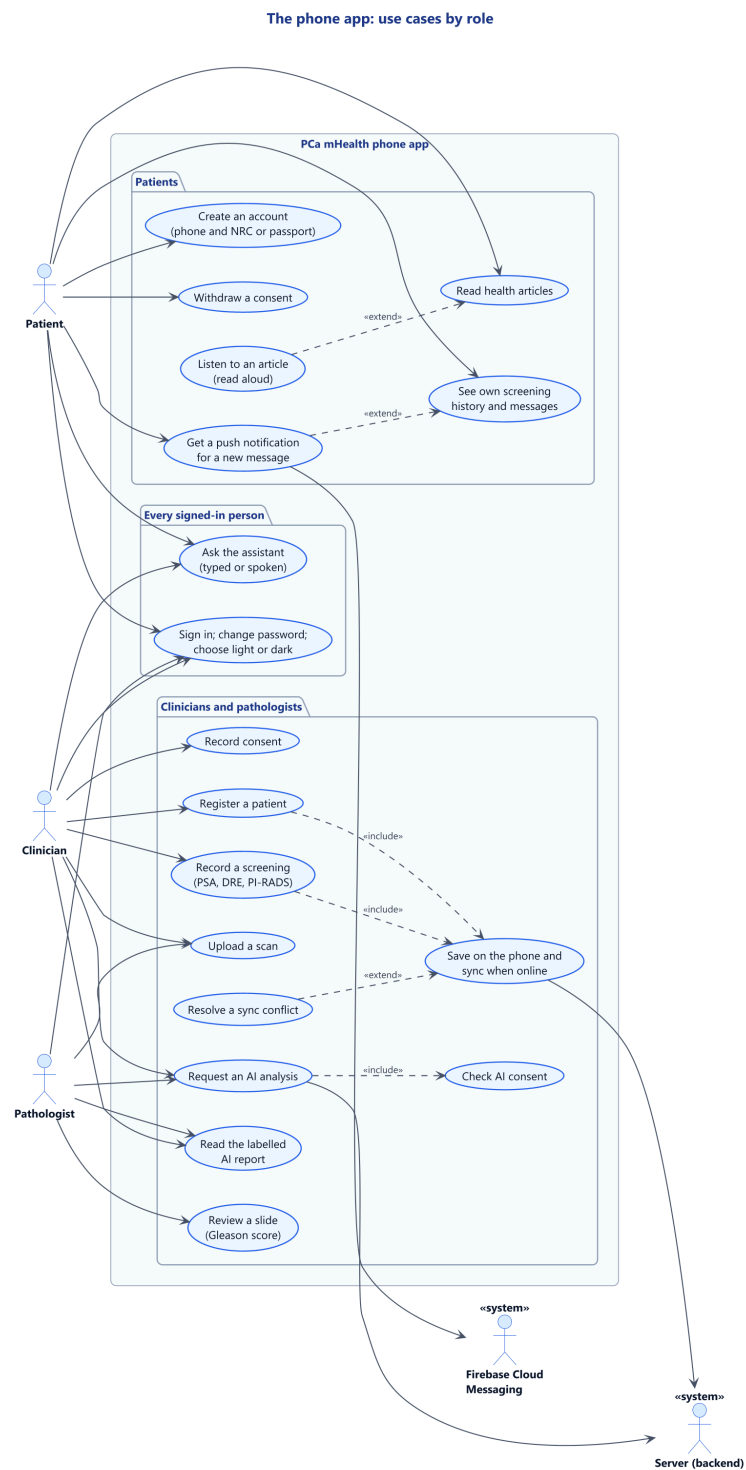
The twelve use cases of the proposal (§3.4) with their actors. Yellow: partly built; grey and dashed: left for later work.



Files: [PNG](#) · [SVG](#) · source [usecase-system.puml](#)

Figure 3.6: Use case diagram: the phone app

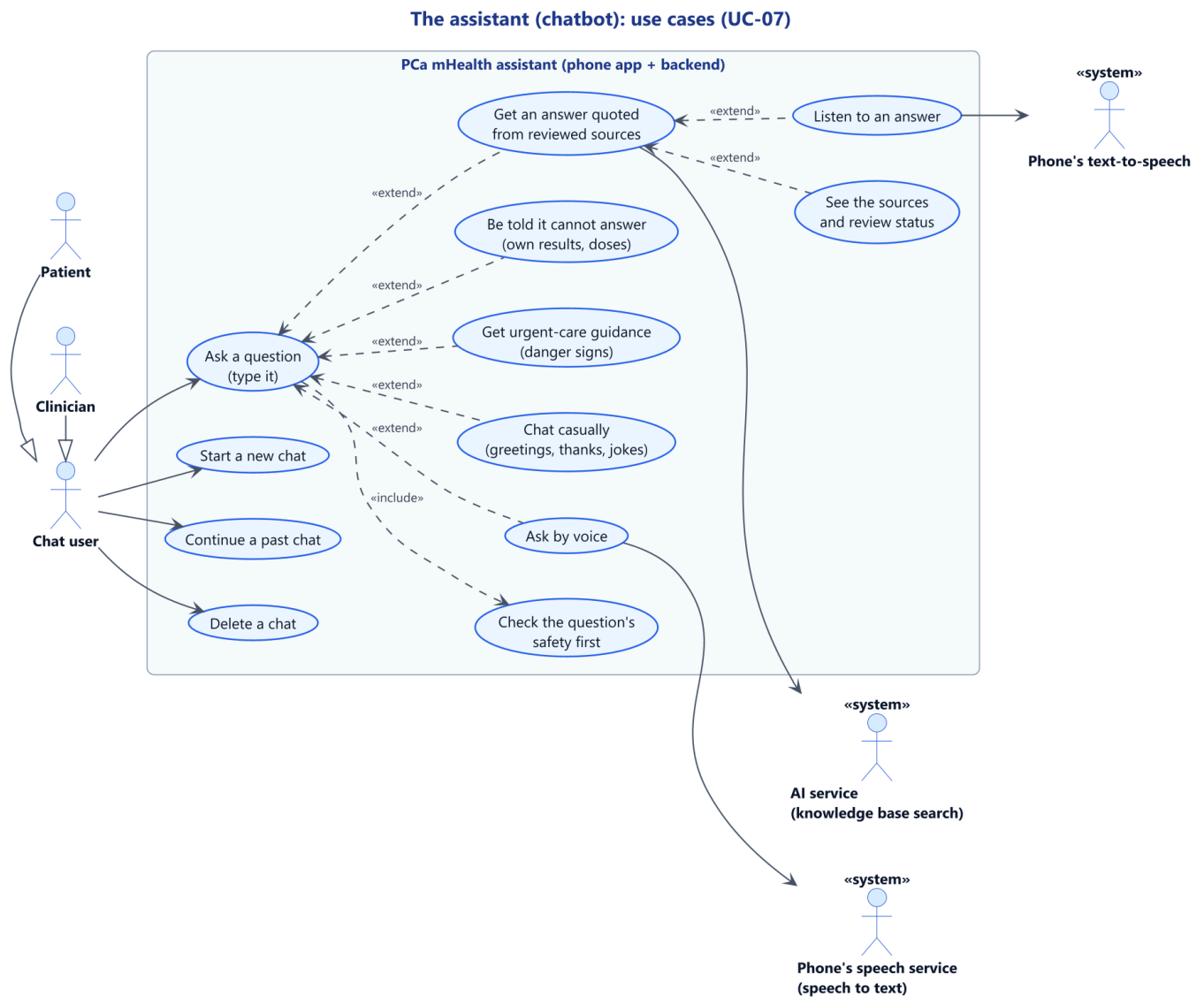
What patients, clinicians and pathologists can do in the app, with the include and extend relations (for example, saving on the phone and syncing is included in registering a patient).



Files: PNG · SVG · source [usecase-mobile-app.puml](#)

Figure 3.7: Use case diagram: the assistant (UC-07)

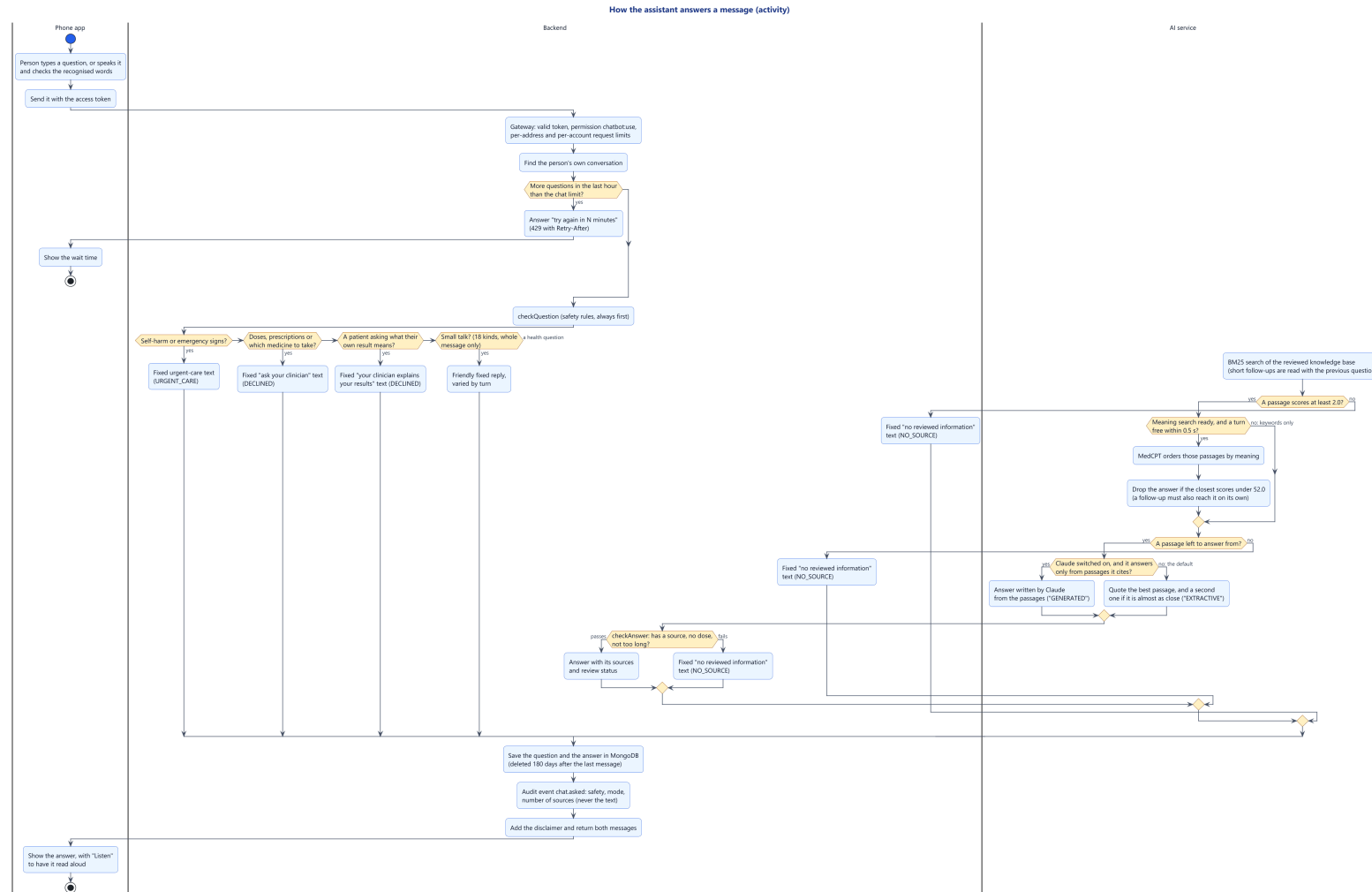
Asking by typing or voice, the safety check that is always included, the kinds of answers, listening to an answer, and managing chats.



Files: [PNG](#) · [SVG](#) · source [usecase-chatbot.puml](#)

Figure 3.8: Activity diagram: how the assistant answers a message

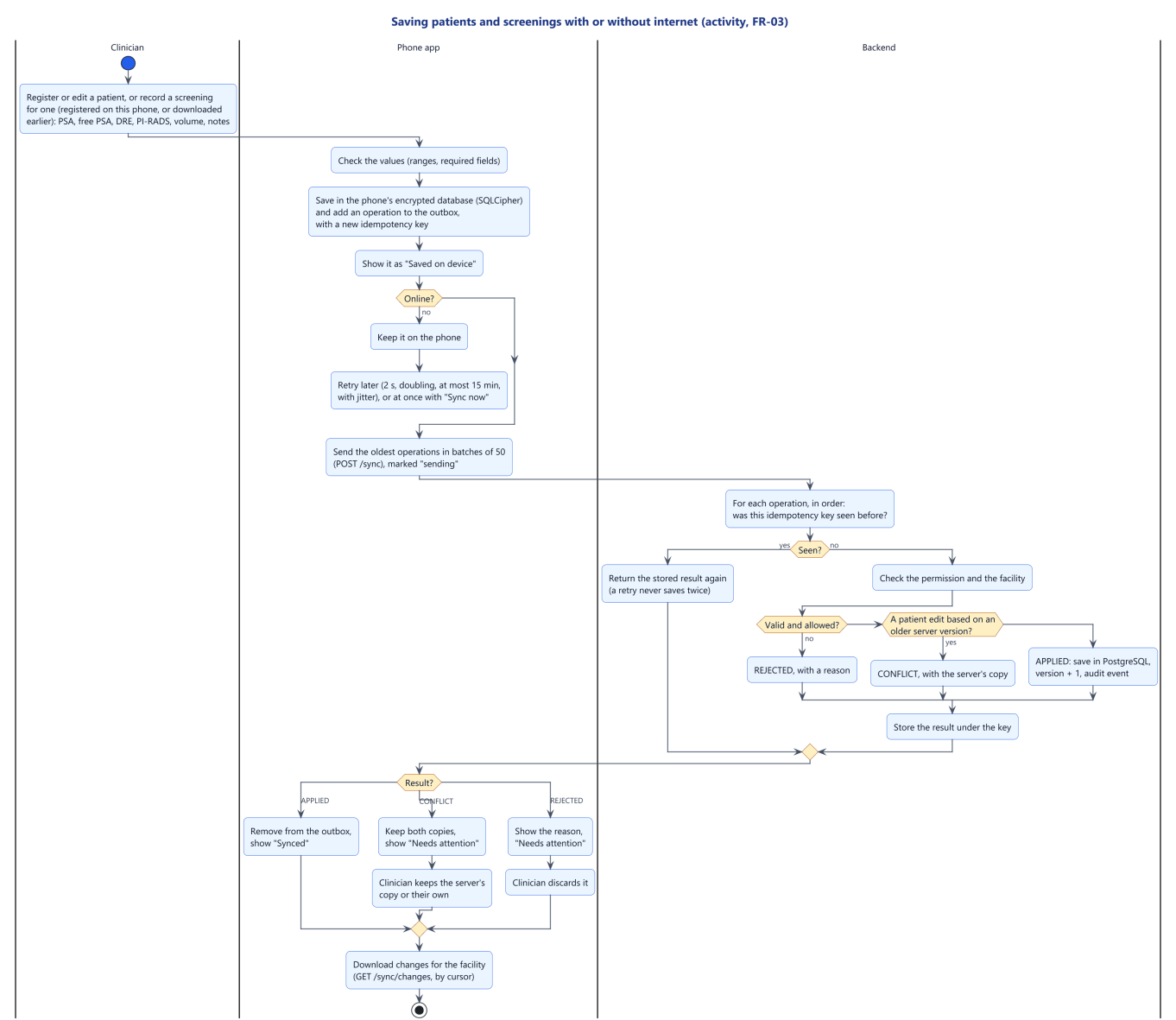
Every decision from the request limit to the stored answer: urgent-care signs, medicines, a patient's own results, small talk, the knowledge-base search (keywords, then meaning) and the answer check.



Files: [PNG](#) · [SVG](#) · source [activity-chat-answer.puml](#)

Figure 3.9: Activity diagram: saving patients and screenings with or without internet

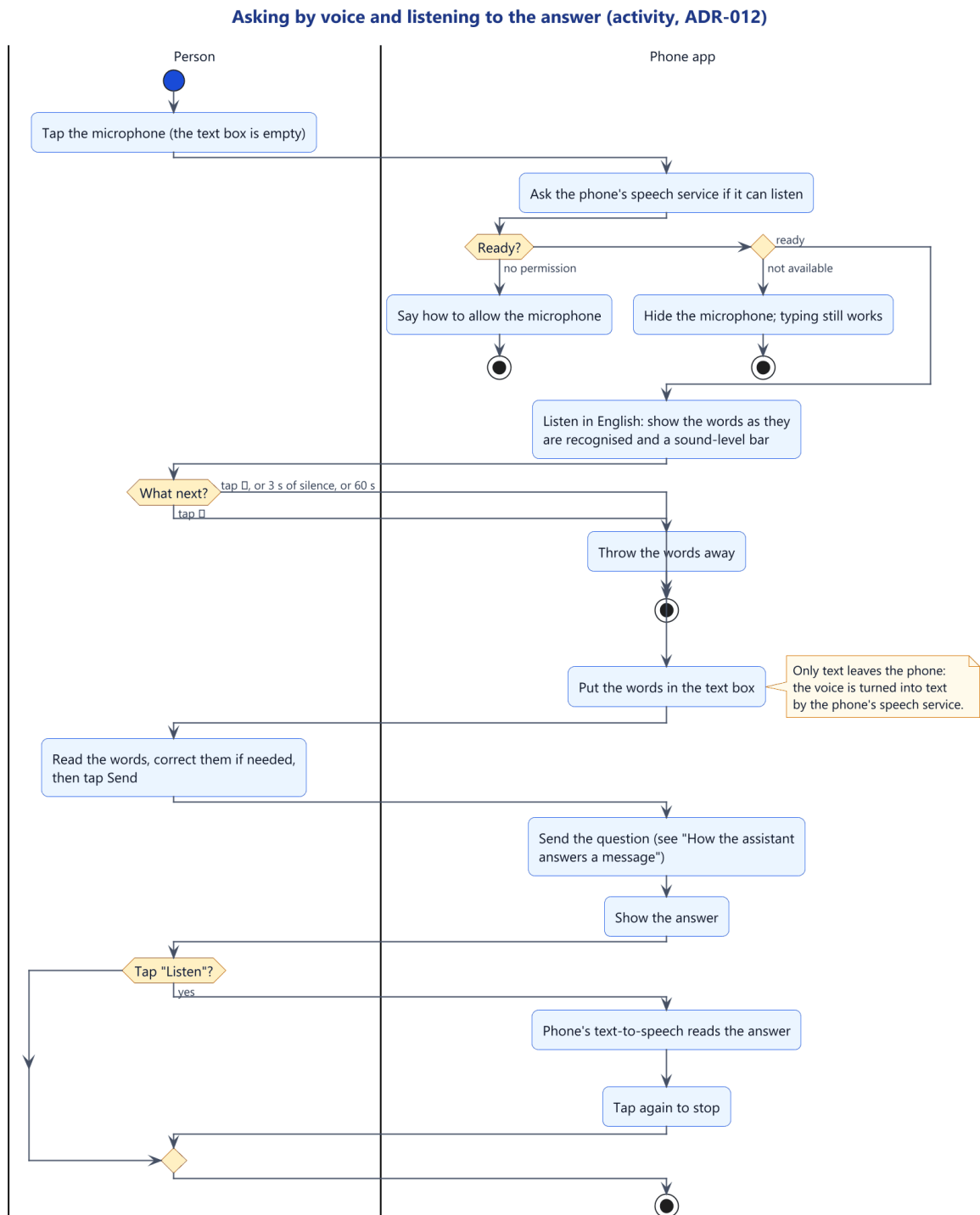
The offline-first design (FR-03): saved on the phone first, sent in batches with idempotency keys, and the three results (applied, conflict, rejected).



Files: PNG · SVG · source activity-offline-capture.puml

Figure 3.10: Activity diagram: asking by voice and listening to the answer

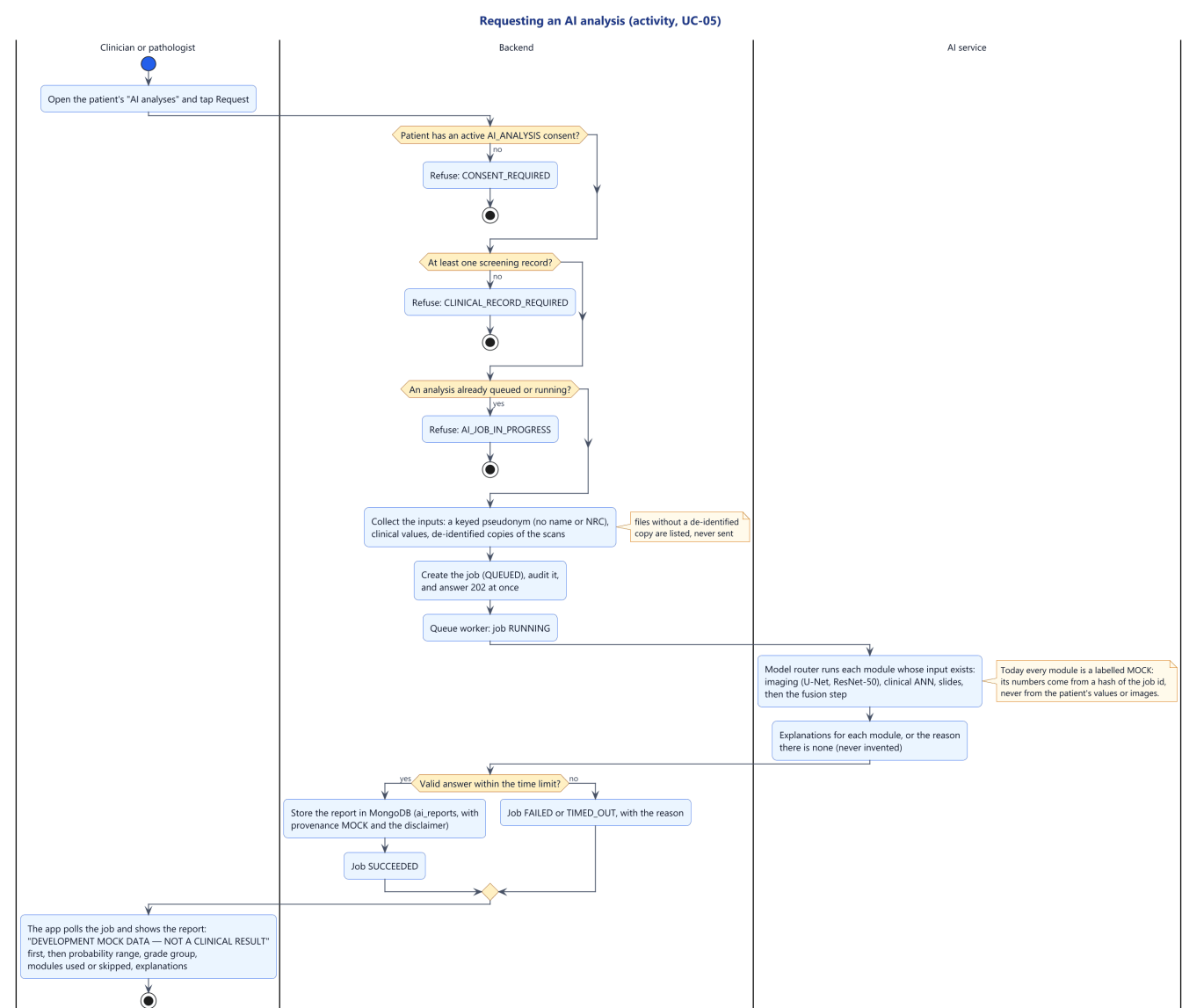
Voice input with the phone's own speech service, and read-aloud (ADR-012).



Files: [PNG](#) · [SVG](#) · source [activity-voice-question.puml](#)

Figure 3.11: Activity diagram: requesting an AI analysis

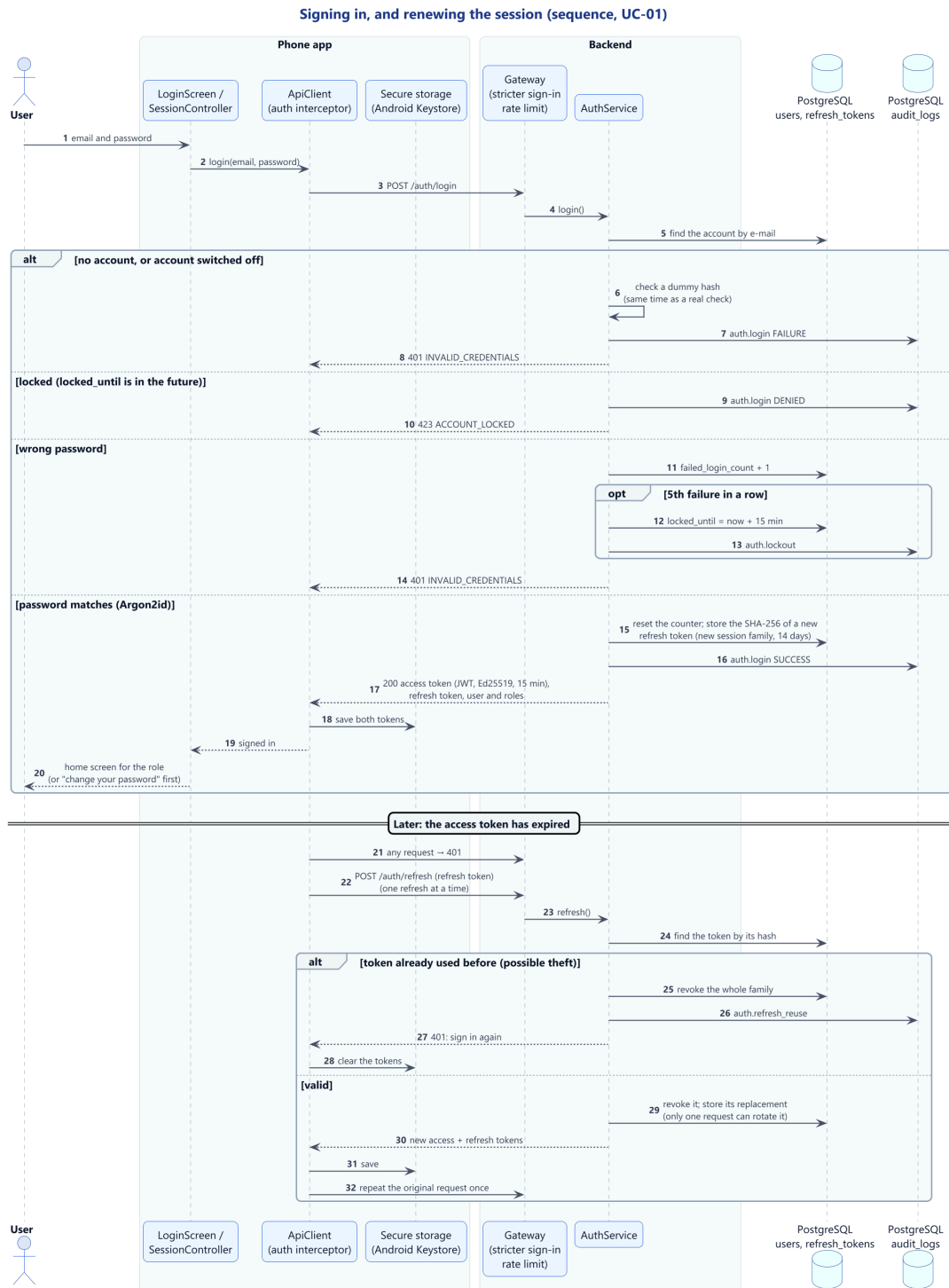
The consent, record and one-at-a-time checks, the de-identified inputs, the queue, and the labelled mock result.



Files: [PNG](#) · [SVG](#) · source [activity-ai-analysis.puml](#)

Figure 3.12: Sequence diagram: signing in and renewing the session

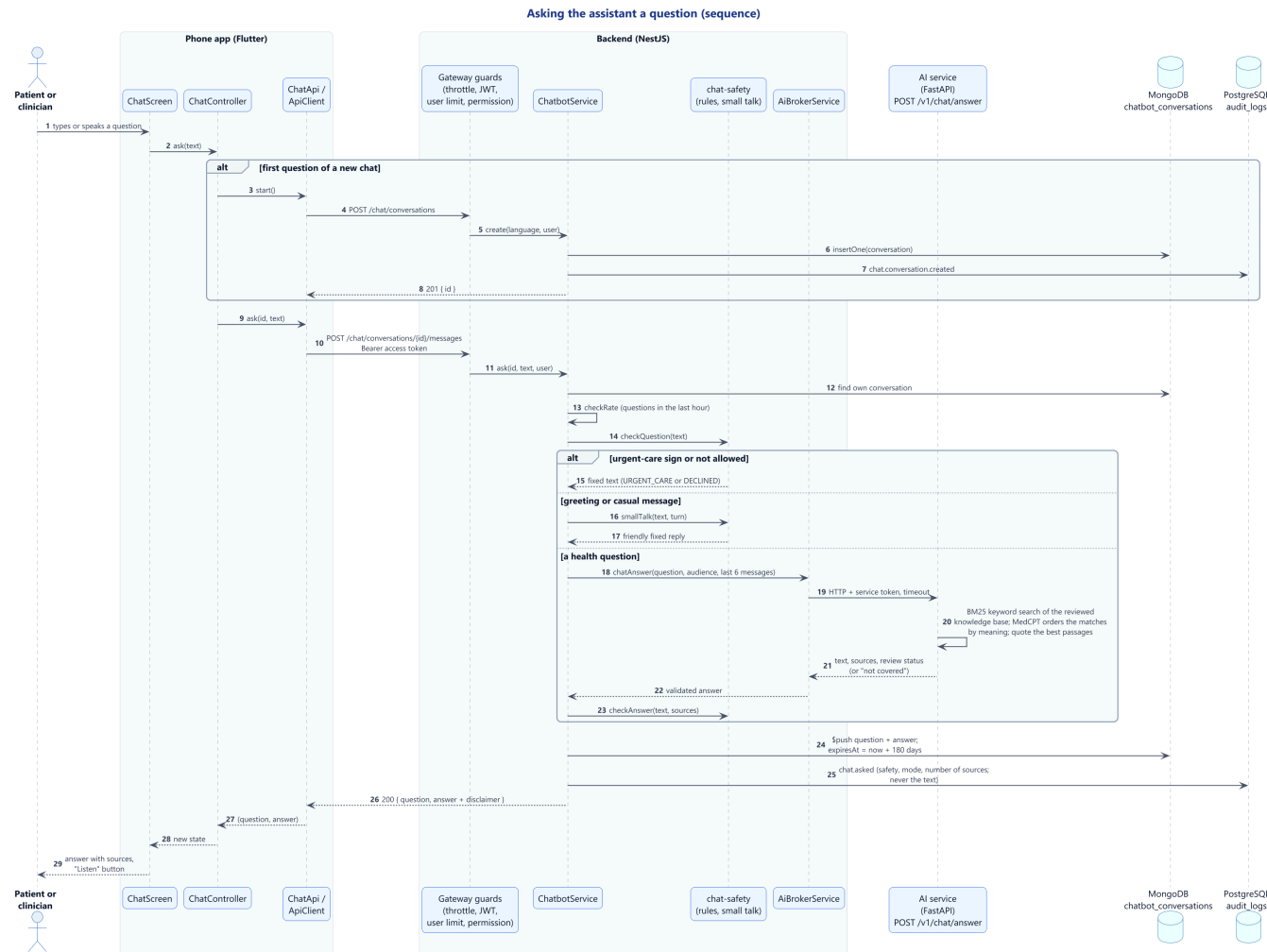
Sign-in with lockout after five failures, the short-lived access token, and refresh-token rotation with theft detection.



Files: PNG · SVG · source [sequence-sign-in.puml](#)

Figure 3.13: Sequence diagram: asking the assistant a question

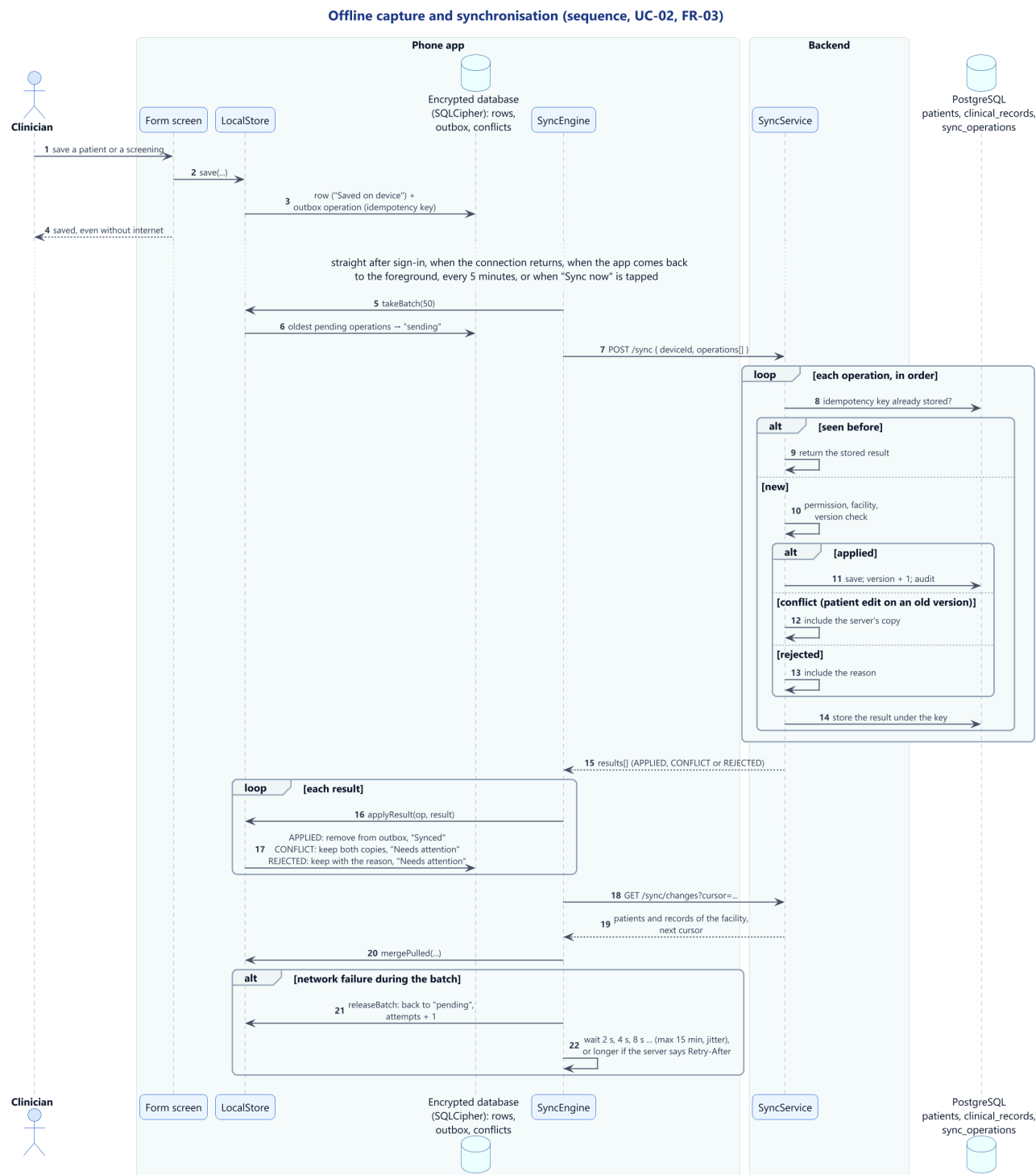
From the chat screen through the gateway guards, the chatbot service and the AI service to MongoDB and the audit log.



Files: [PNG](#) · [SVG](#) · source [sequence-chat.puml](#)

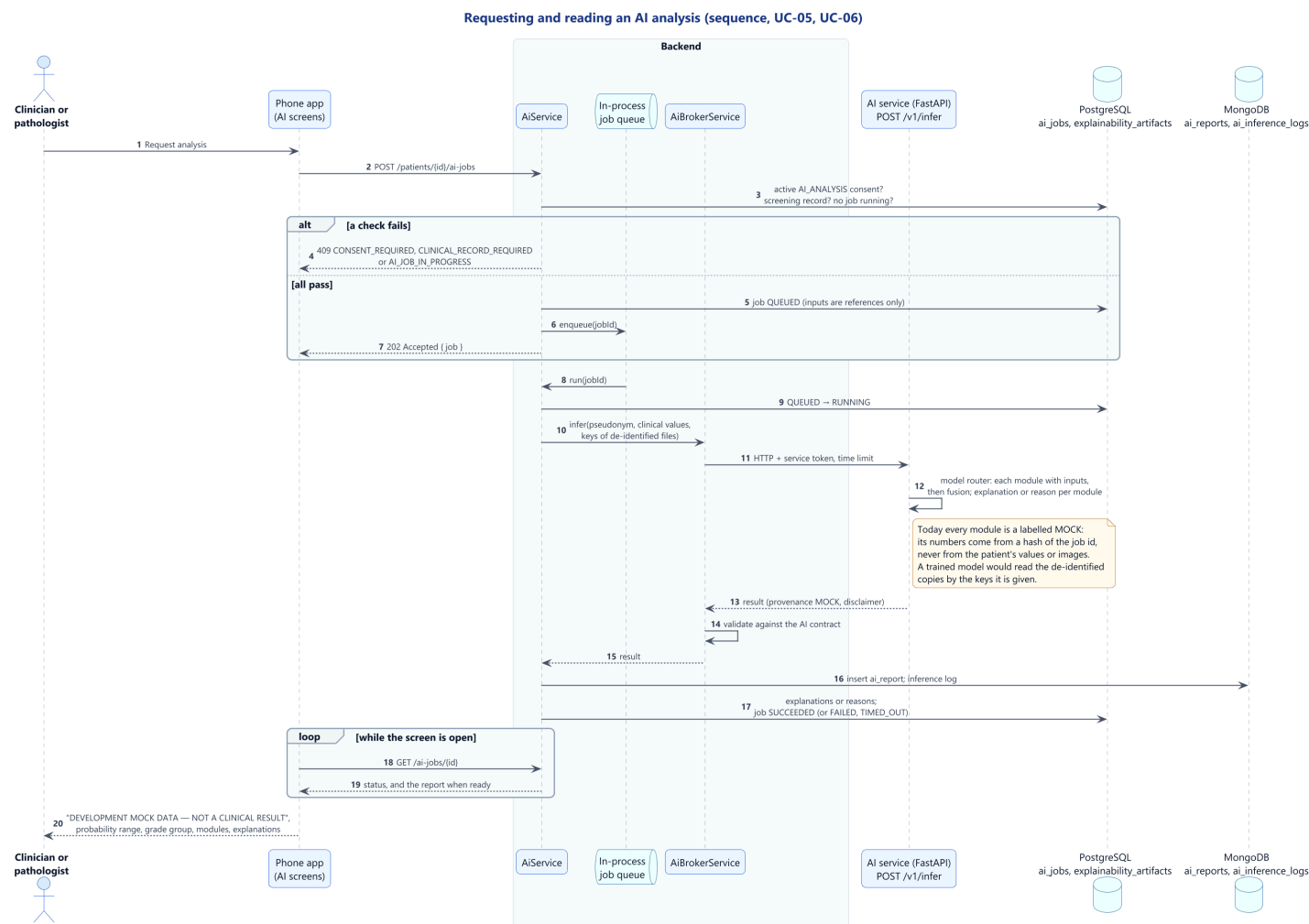
Figure 3.14: Sequence diagram: offline capture and synchronisation

The outbox on the phone, the batch sent to the server, the per-operation results, and downloading the facility's changes.



Files: PNG · SVG · source [sequence-offline-sync.puml](#)

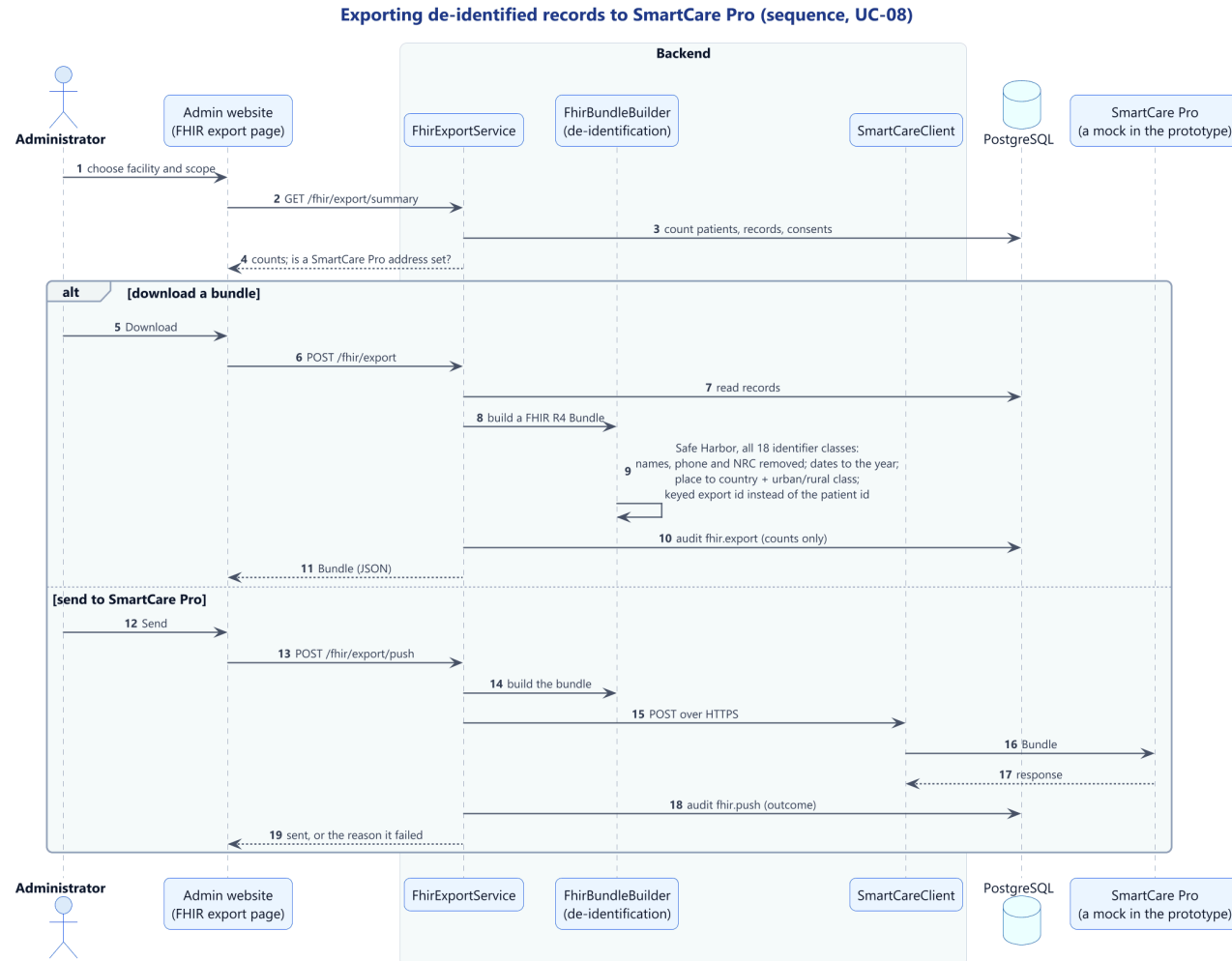
Figure 3.15: Sequence diagram: requesting and reading an AI analysis



Files: [PNG](#) · [SVG](#) · source [sequence-ai-analysis.puml](#)

Figure 3.16: Sequence diagram: exporting de-identified records to SmartCare Pro

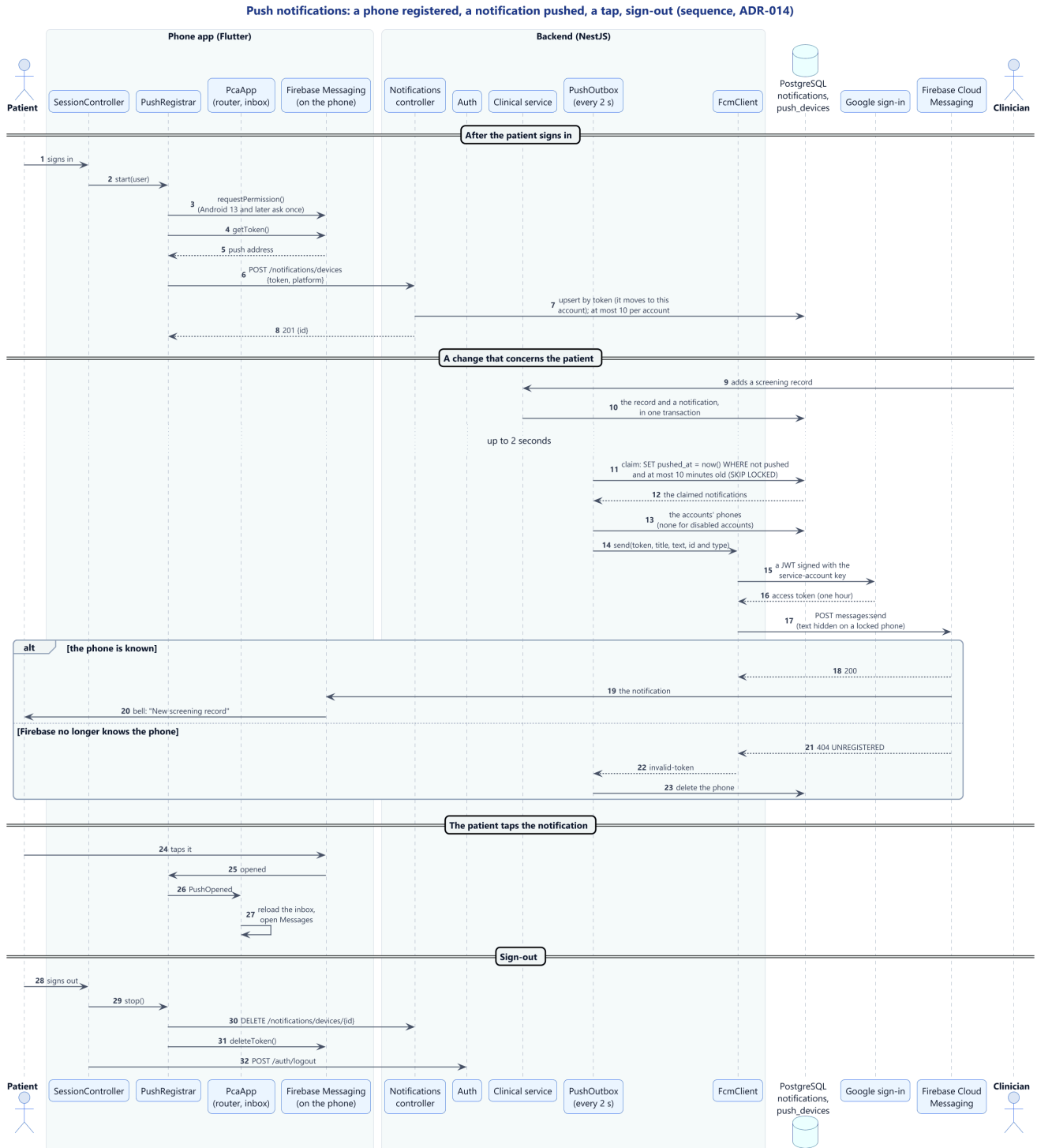
The FHIR R4 export (UC-08) with Safe Harbor de-identification of all 18 identifier classes.



Files: [PNG](#) · [SVG](#) · source [sequence-fhir-export.puml](#)

Figure 3.17: Sequence diagram: push notifications

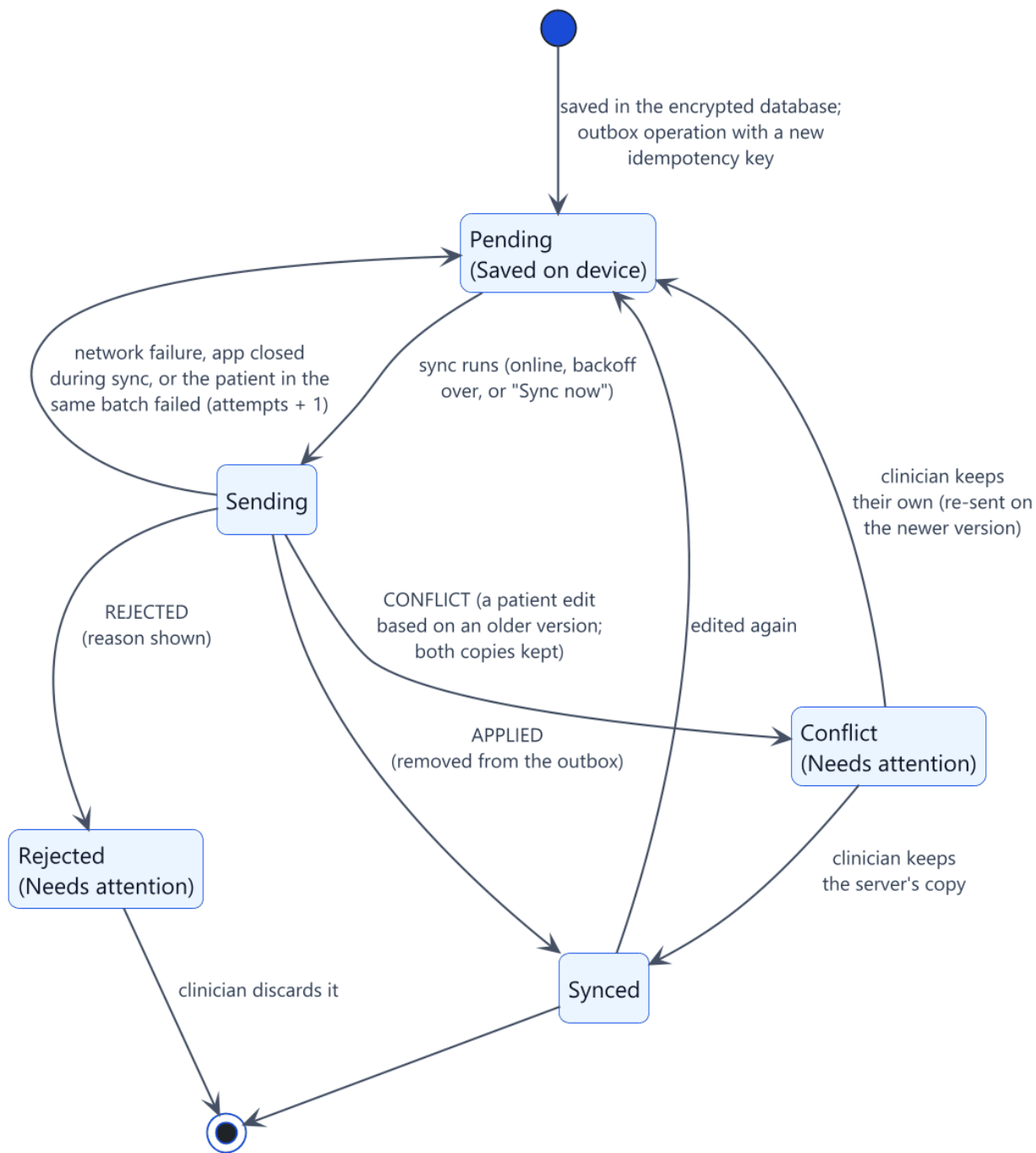
A patient's phone is registered after sign-in; a new notification is claimed by the outbox and sent through Firebase Cloud Messaging (text only, hidden on a locked phone); a tap opens the messages; sign-out removes the phone (ADR-014). Push is switched on with the owner's Firebase project.



Files: [PNG](#) · [SVG](#) · source [sequence-push.puml](#)

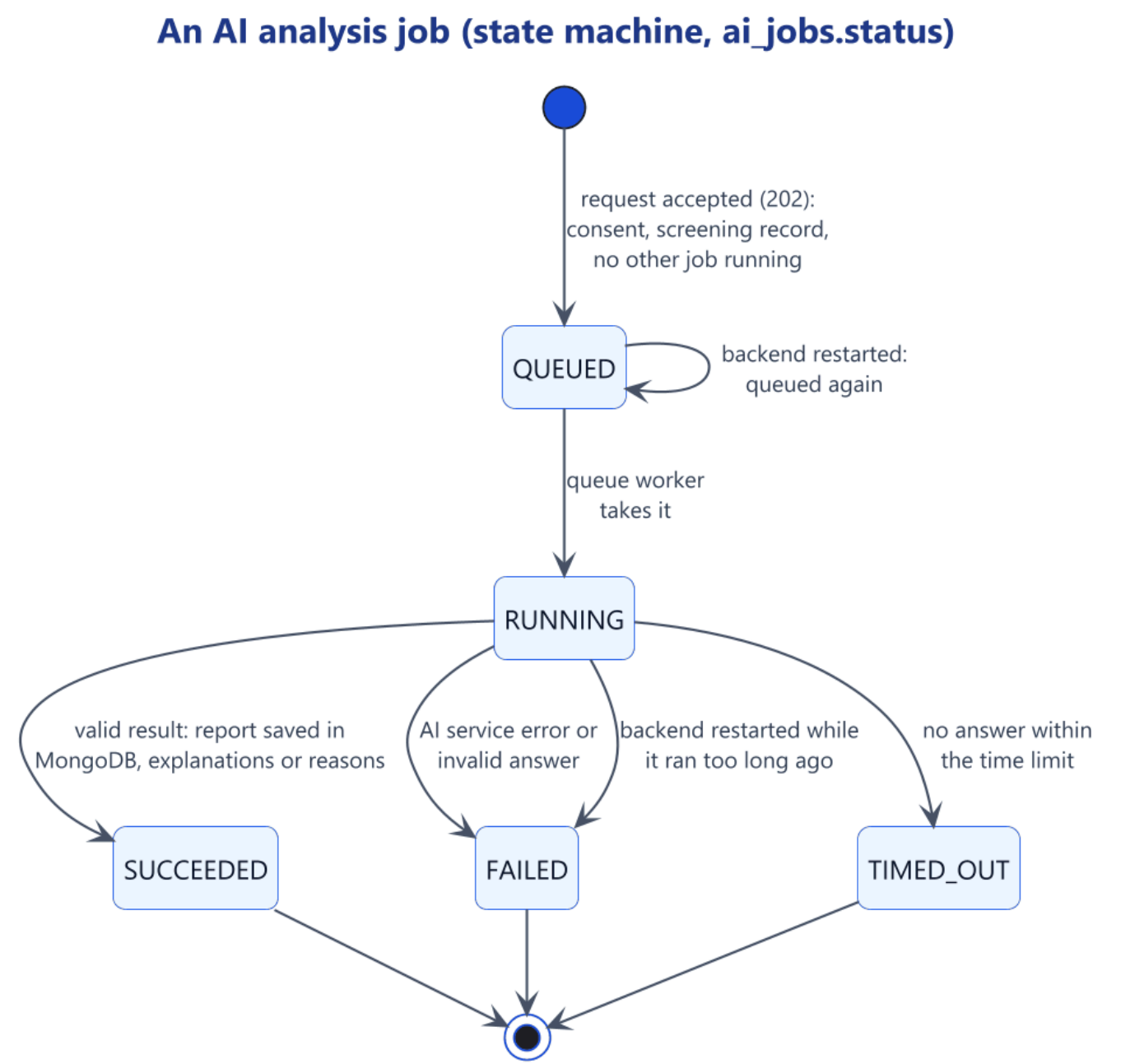
Figure 3.18: State diagram: a change saved on the phone

A change saved on the phone: its states until the server has it (state machine)



Files: [PNG](#) · [SVG](#) · source [state-offline-change.puml](#)

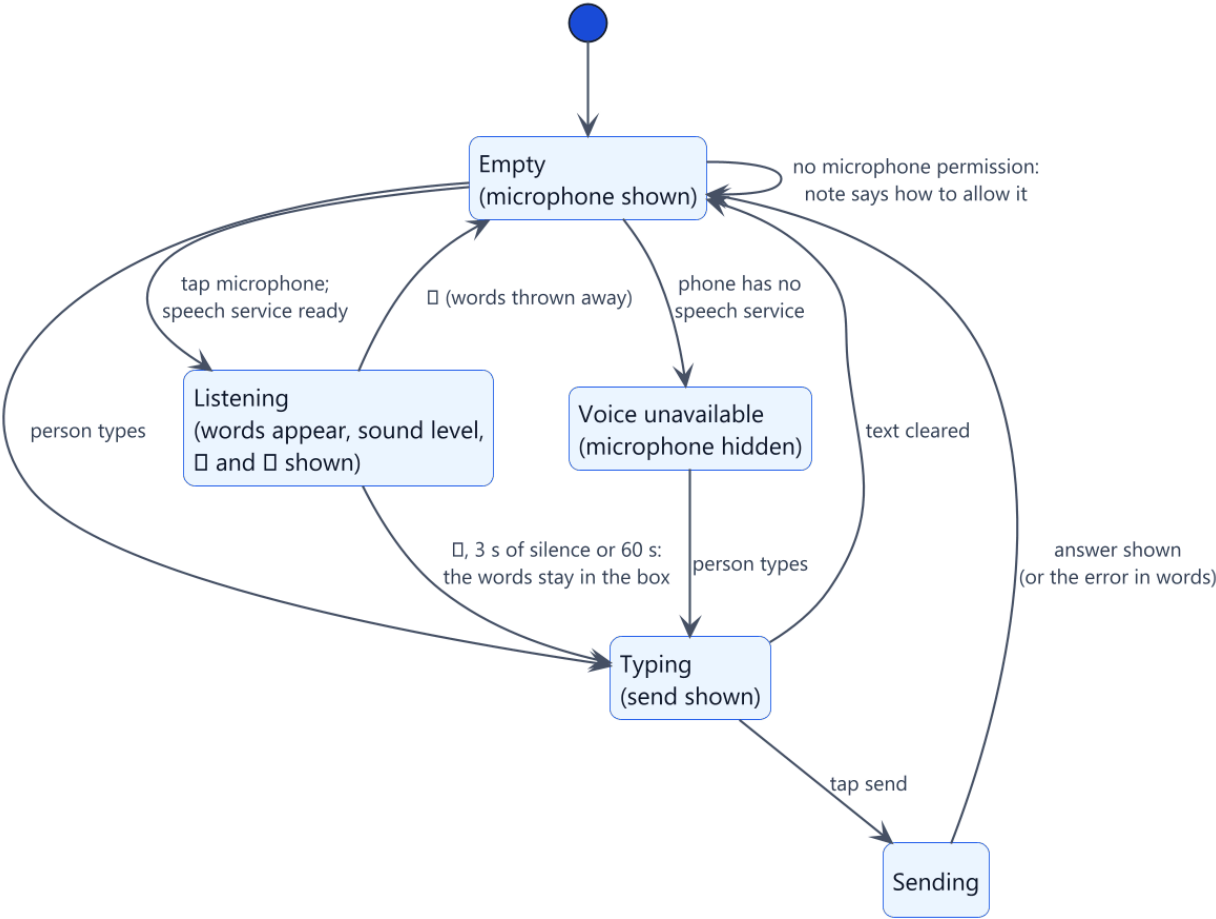
Figure 3.19: State diagram: an AI analysis job



Files: [PNG](#) · [SVG](#) · source [state-ai-job.puml](#)

Figure 3.20: State diagram: the chat's message box with voice input

The chat's message box with voice input (state machine, ADR-012)

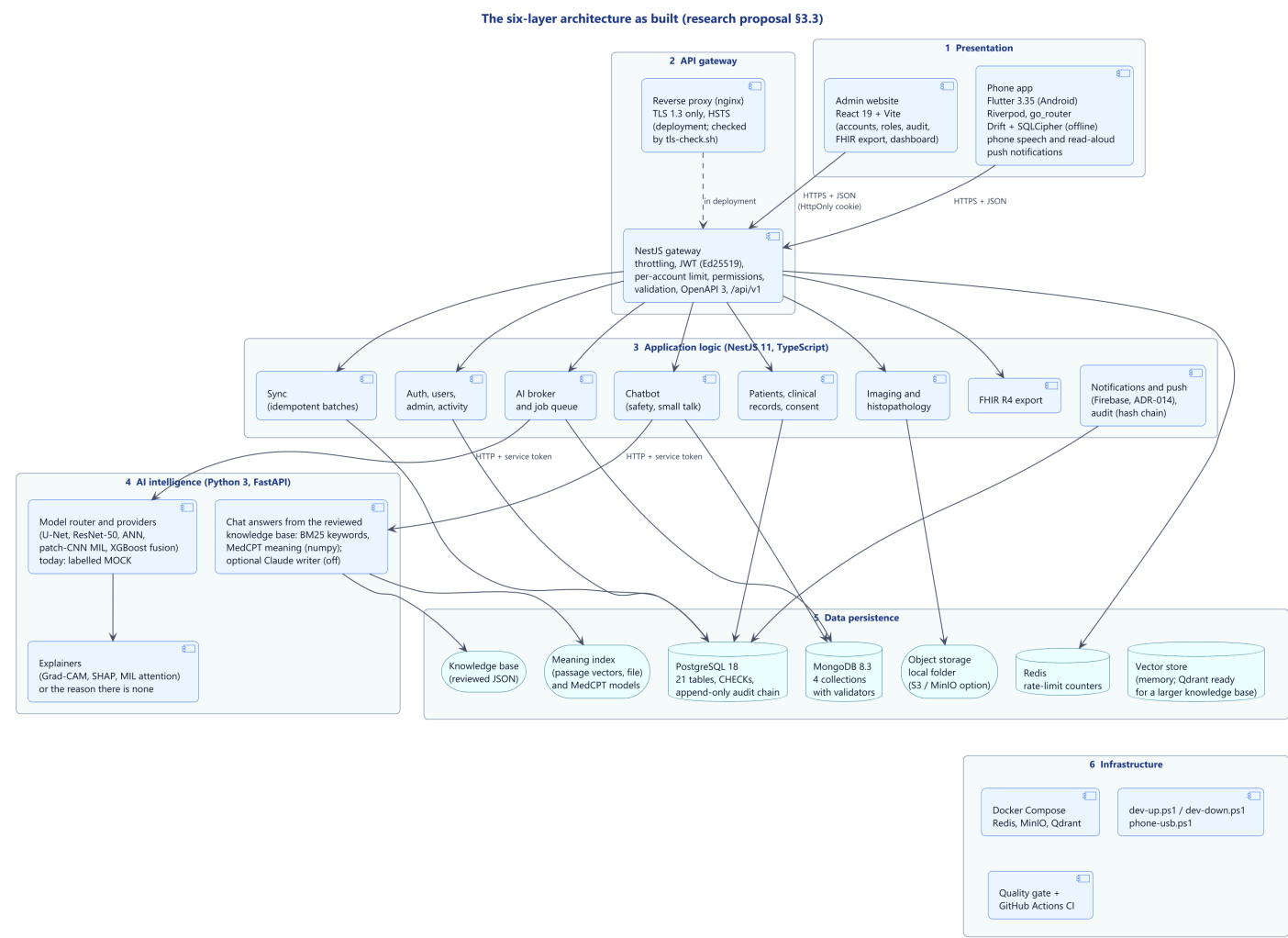


Files: [PNG](#) · [SVG](#) · source [state-voice-input.puml](#)

3.5 Program design, database design and interface design

Figure 3.21: The six-layer architecture as built

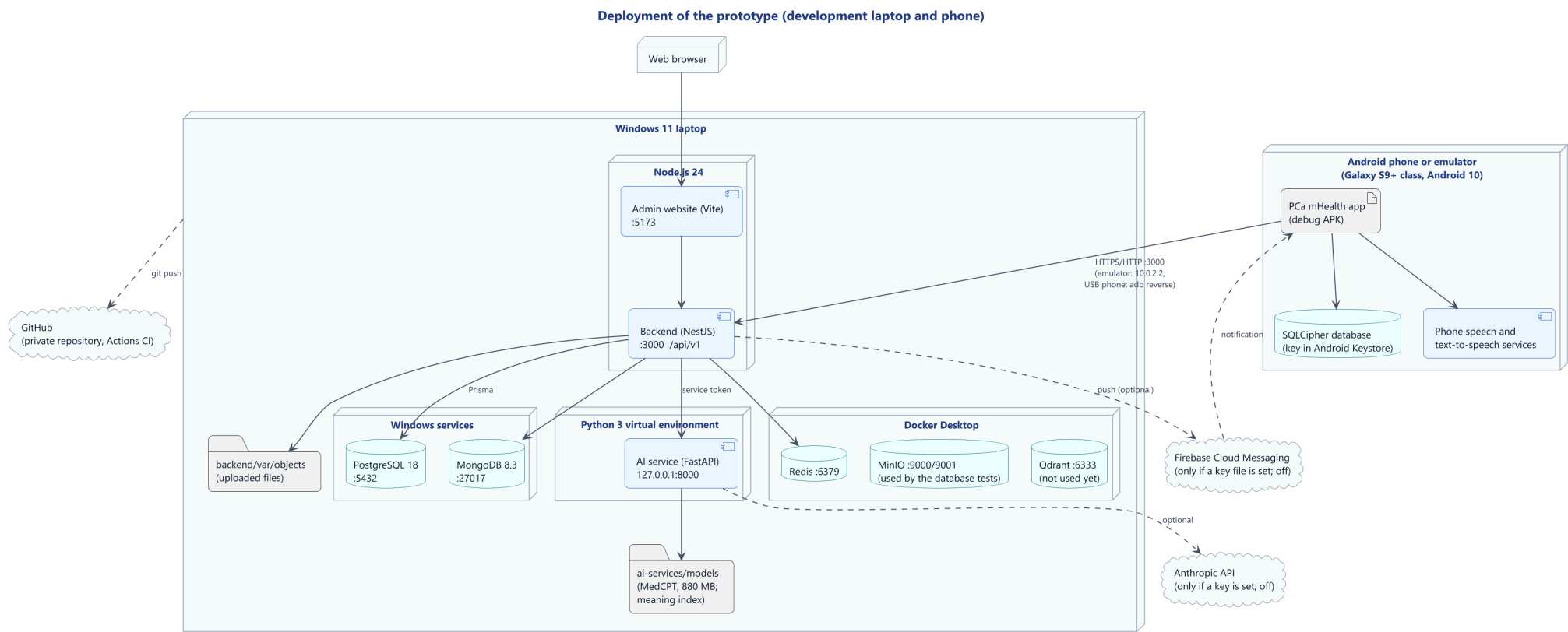
The proposal's six layers (§3.3) with what each contains in the prototype.



Files: [PNG](#) · [SVG](#) · source [architecture-layers.puml](#)

Figure 3.22: Deployment diagram

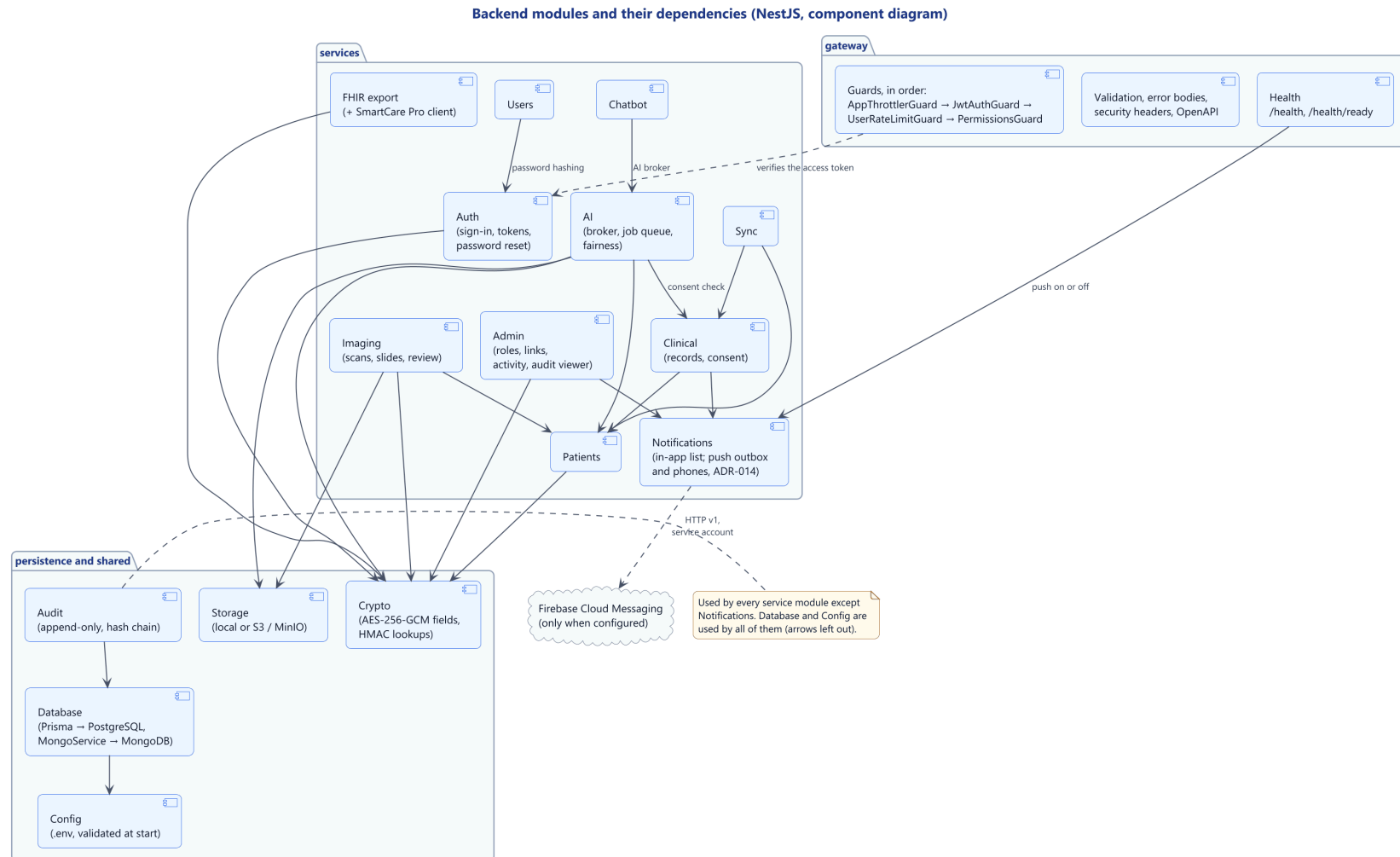
Where each part runs: the phone or emulator, the laptop's Windows services, Node.js, Python and Docker, and the outside services.



Files: [PNG](#) · [SVG](#) · source [deployment.puml](#)

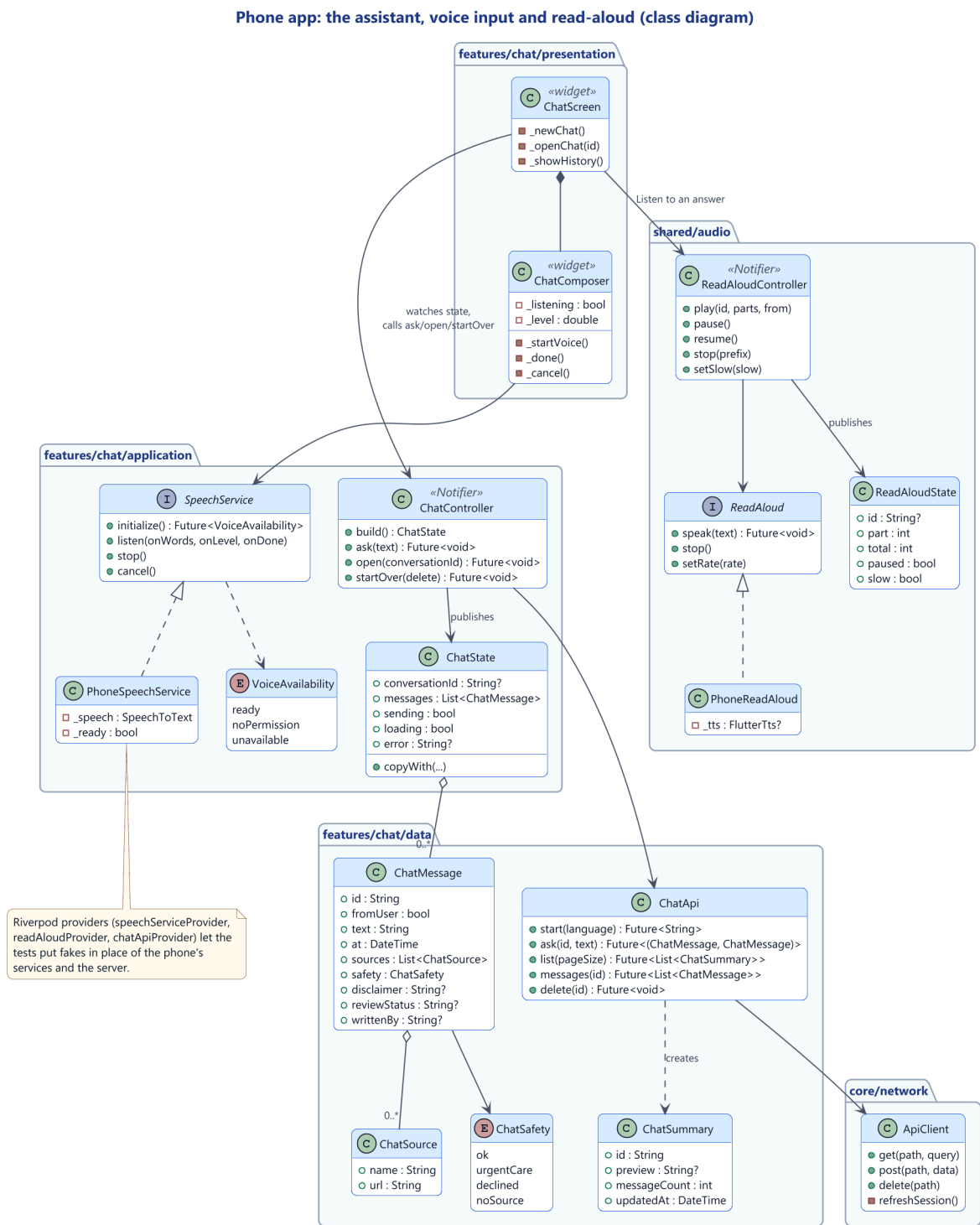
Figure 3.23: Component diagram: the backend modules

The NestJS modules and their real dependencies (taken from the services' constructors), and the order of the gateway guards.



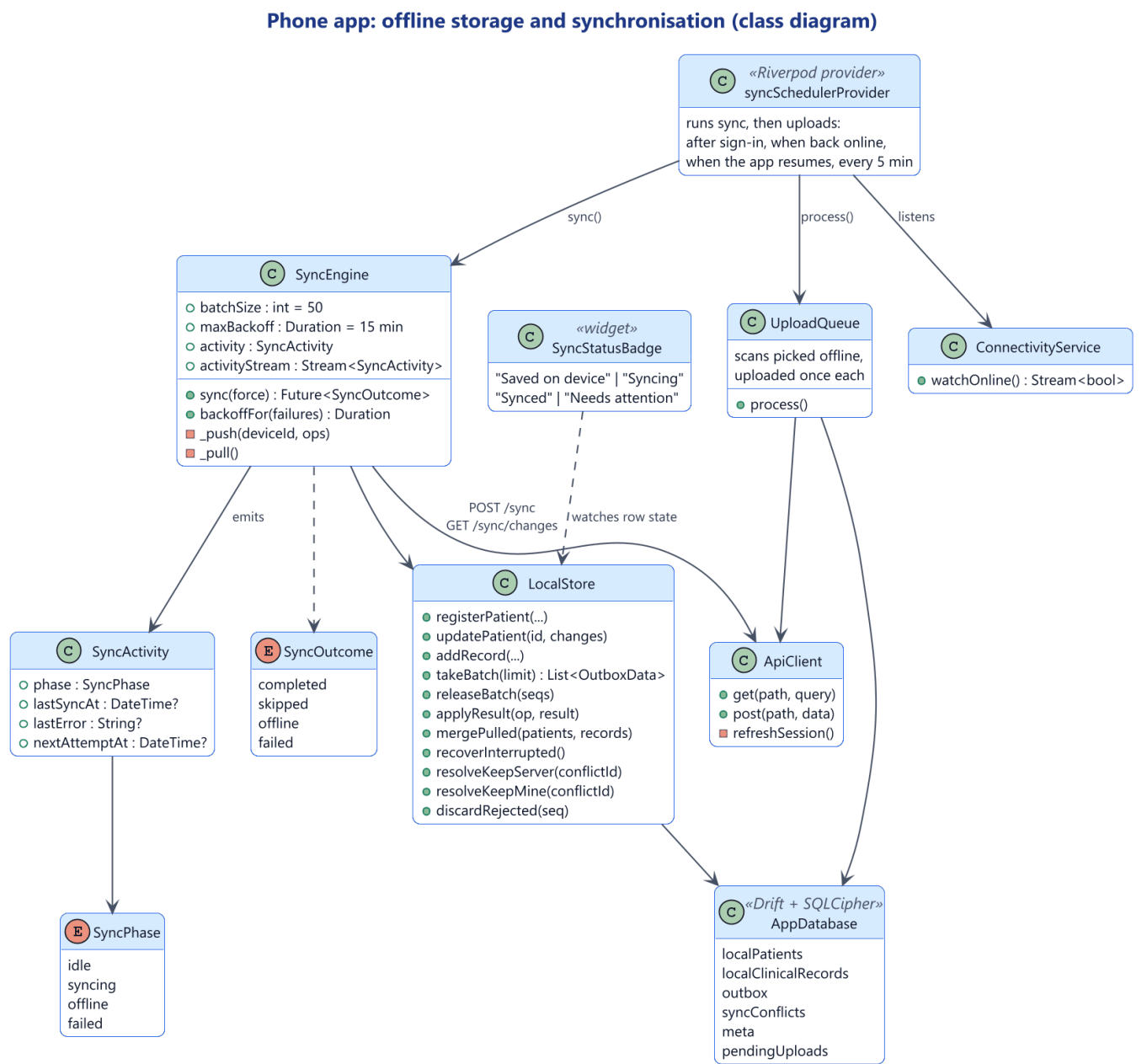
Files: [PNG](#) · [SVG](#) · source [component-backend.puml](#)

Figure 3.24: Class diagram: the app's assistant, voice input and read-aloud



Files: PNG · SVG · source [class-app-chat-audio.puml](#)

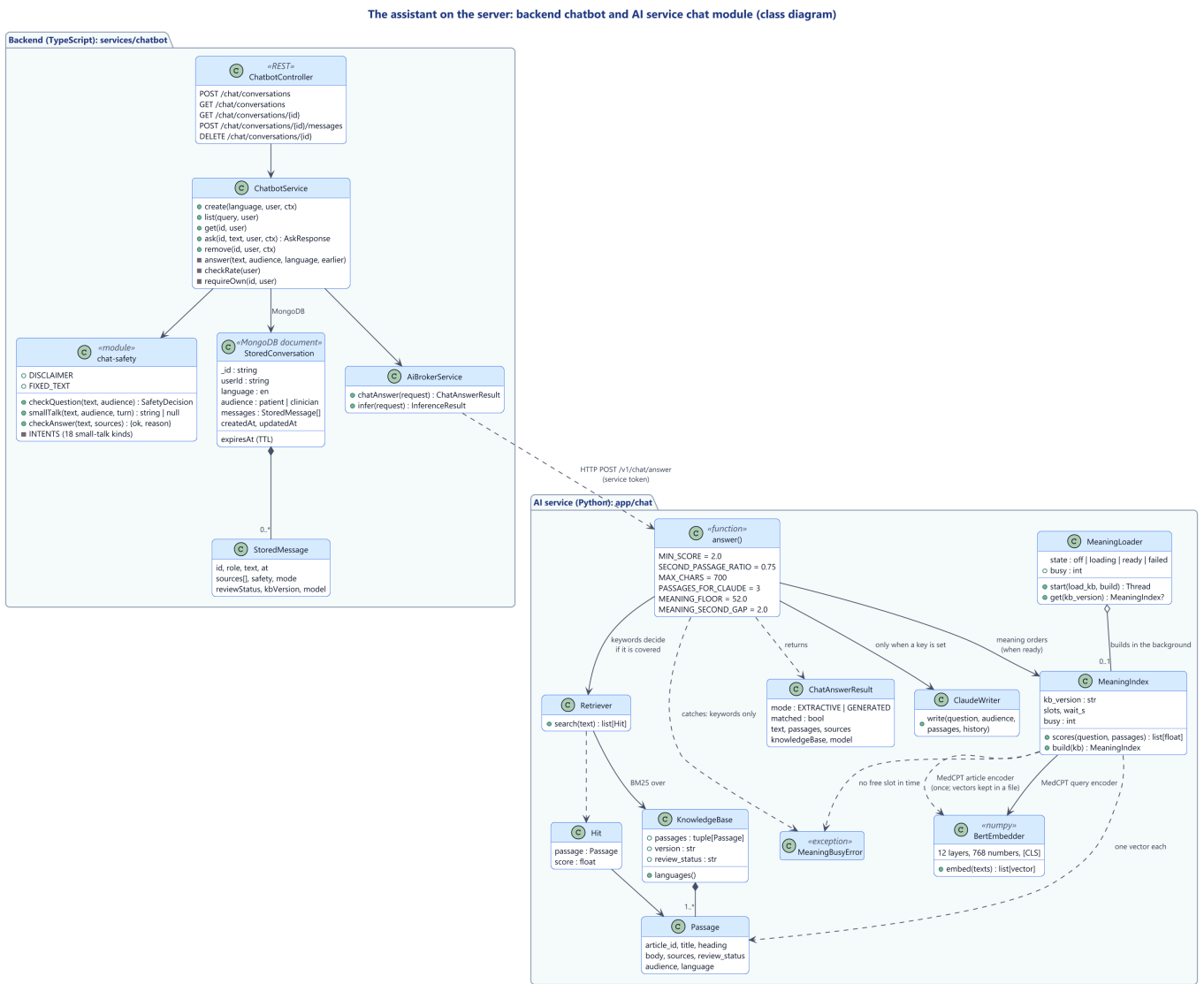
Figure 3.25: Class diagram: the app's offline storage and synchronisation



Files: [PNG](#) · [SVG](#) · source [class-app-offline.puml](#)

Figure 3.26: Class diagram: the assistant on the server

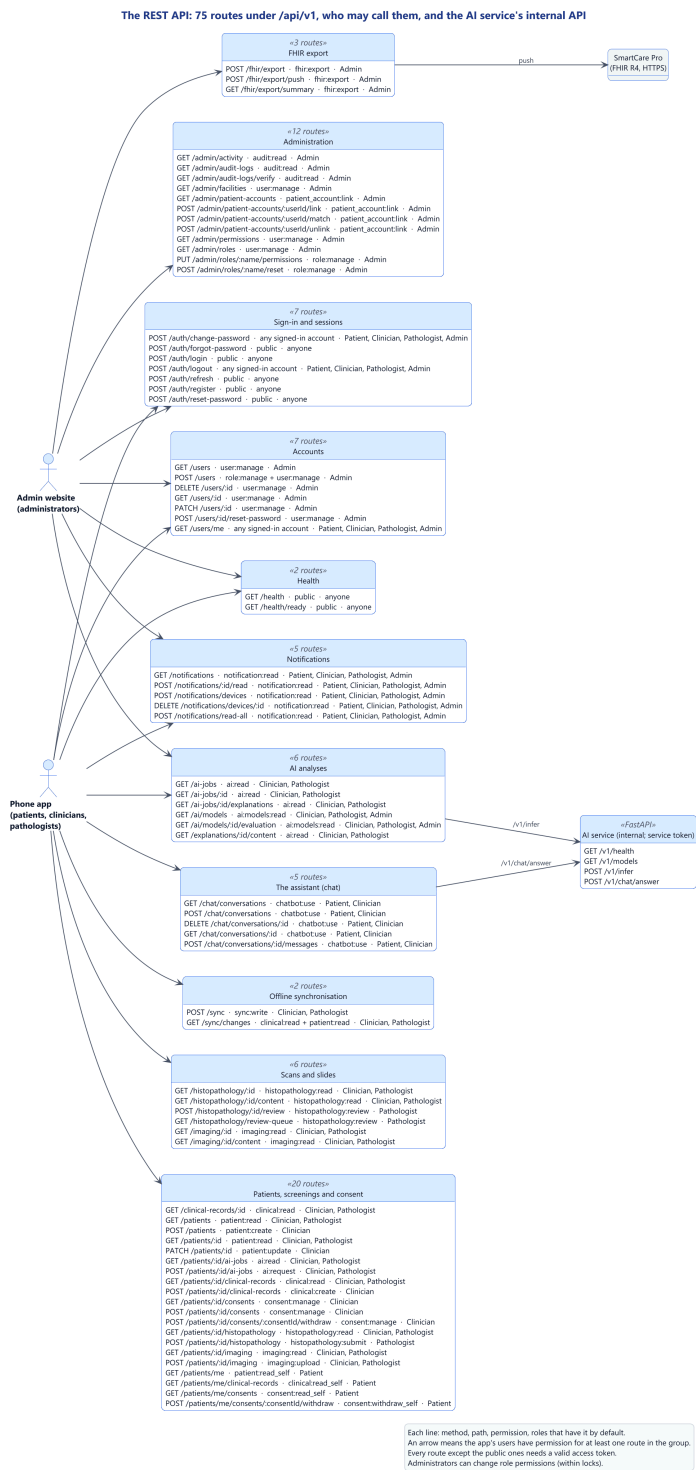
The backend's chatbot module and the AI service's chat module (keyword retrieval, the MedCPT meaning search, knowledge base, optional Claude writer).



Files: PNG · SVG · source [class-chat-server.puml](#)

Figure 3.27: API diagram: every route, its permission and its default roles

All REST routes under `/api/v1`, grouped by resource, with the apps whose users may call them, and the AI service's internal API. Generated from [access-matrix.md](#).



Files: PNG · SVG · source generated: [api-map.puml](#)

Figure 3.28: The API explorer (OpenAPI 3, Swagger UI)

The backend describes its own API at `http://localhost:3000/api/docs` (screenshot of the running backend).

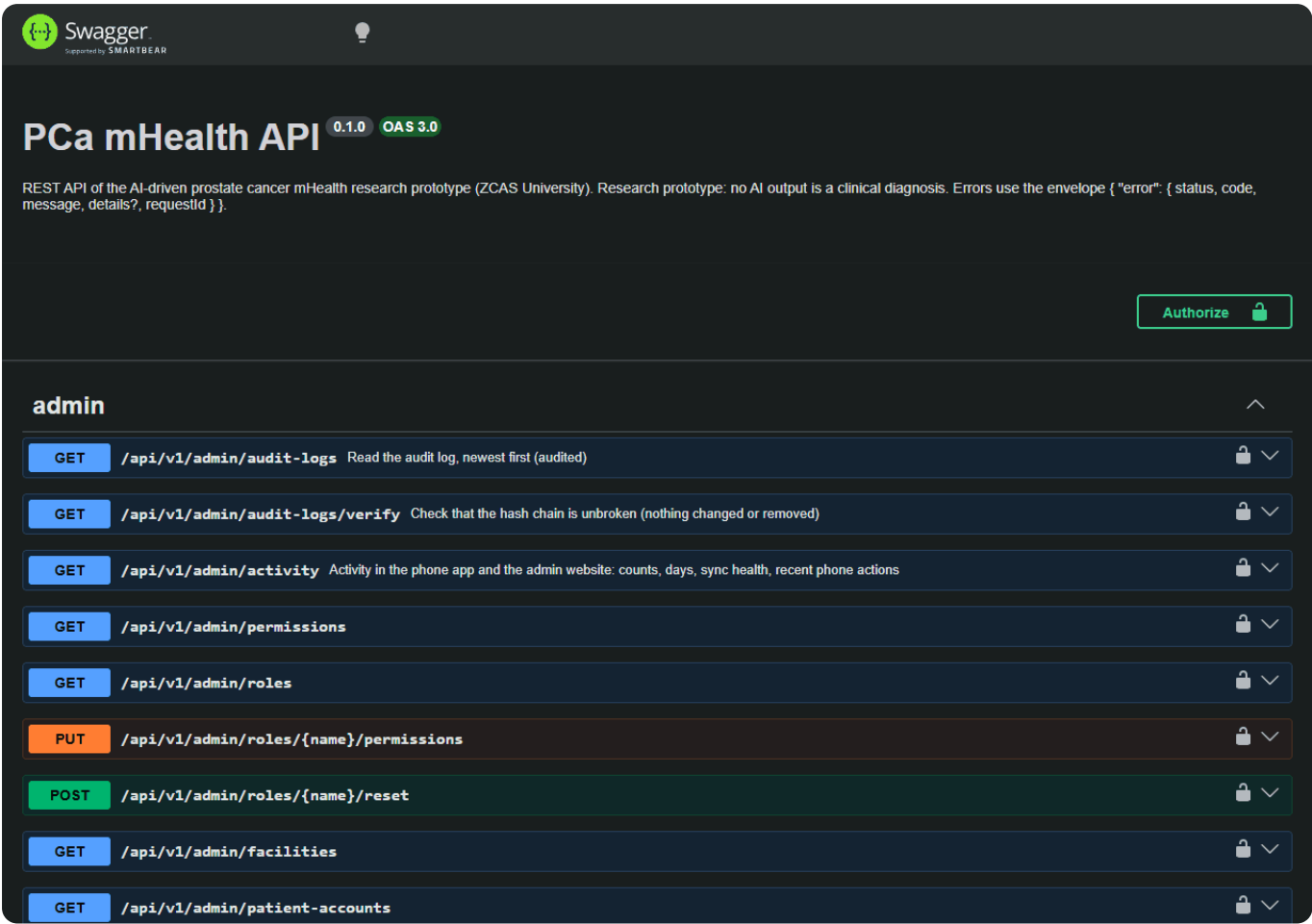
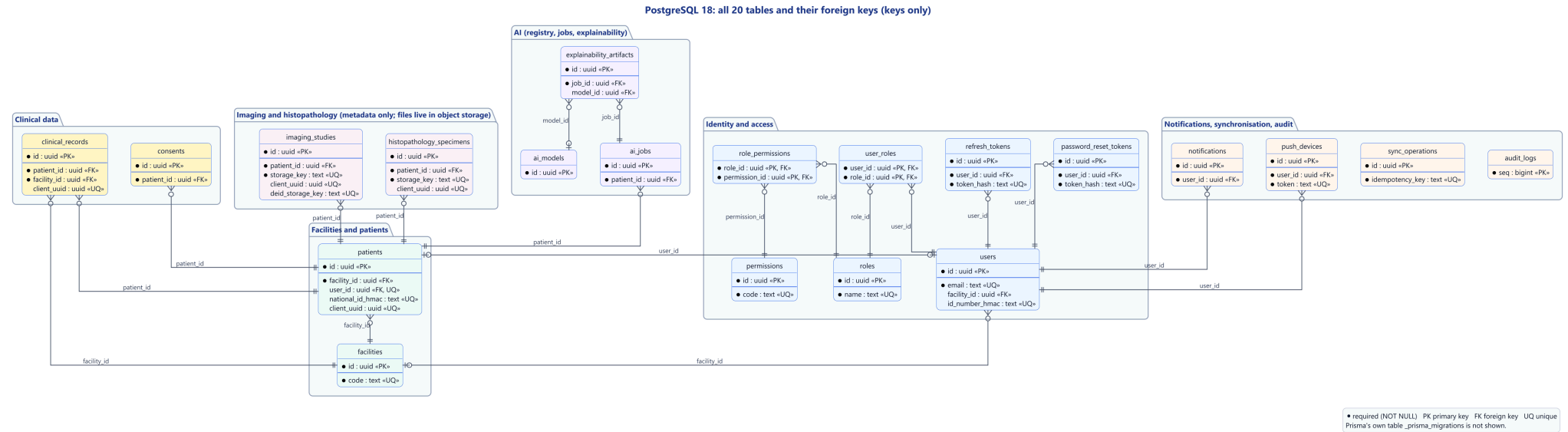


Figure 3.29: Entity-relationship diagram: all PostgreSQL tables and their keys

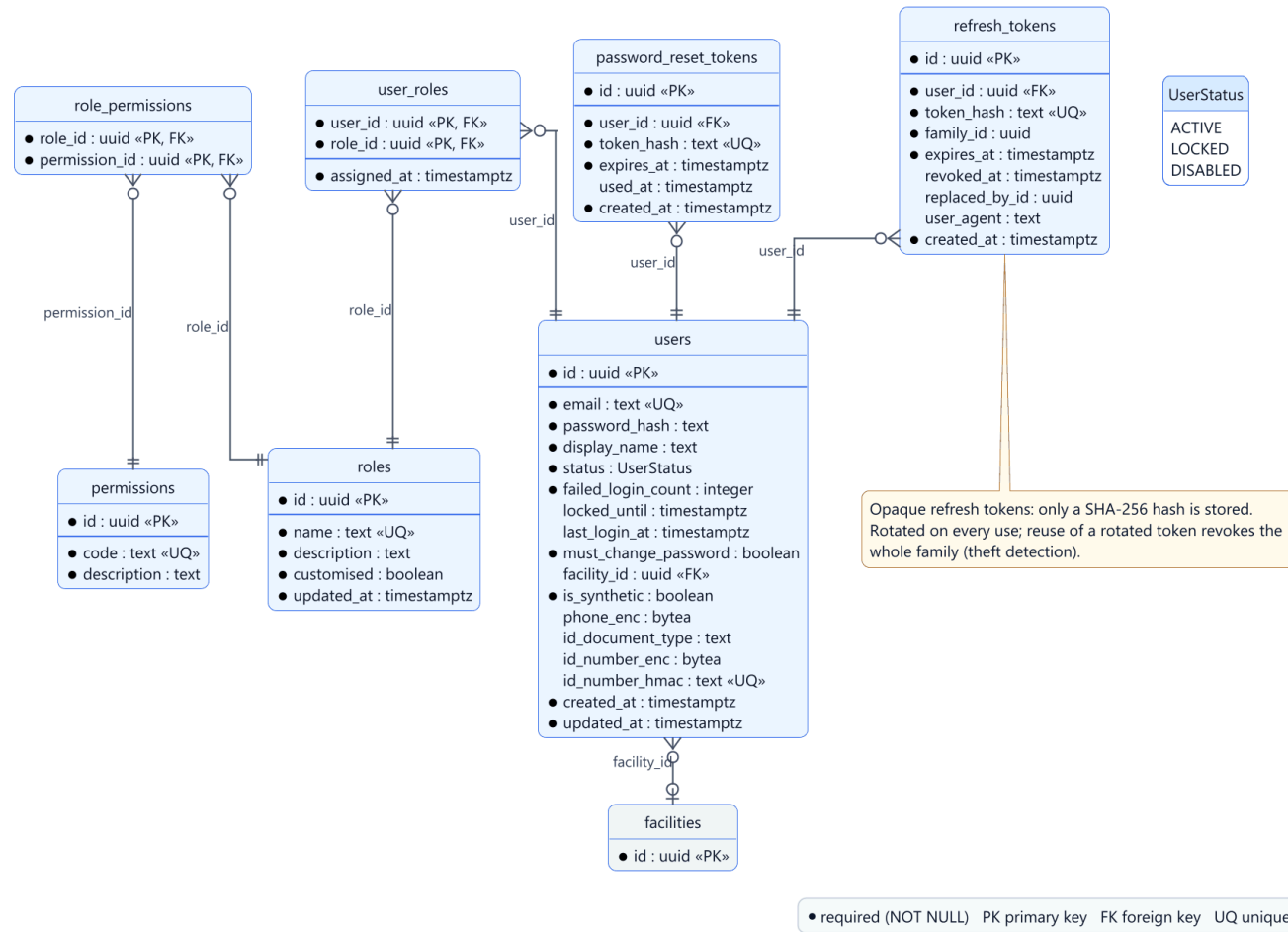
Generated from `schema.prisma`. Crow's-foot notation: a circle means "zero", a bar "one", a fork "many".



Files: [PNG](#) · [SVG](#) · source generated: [erd-overview.puml](#)

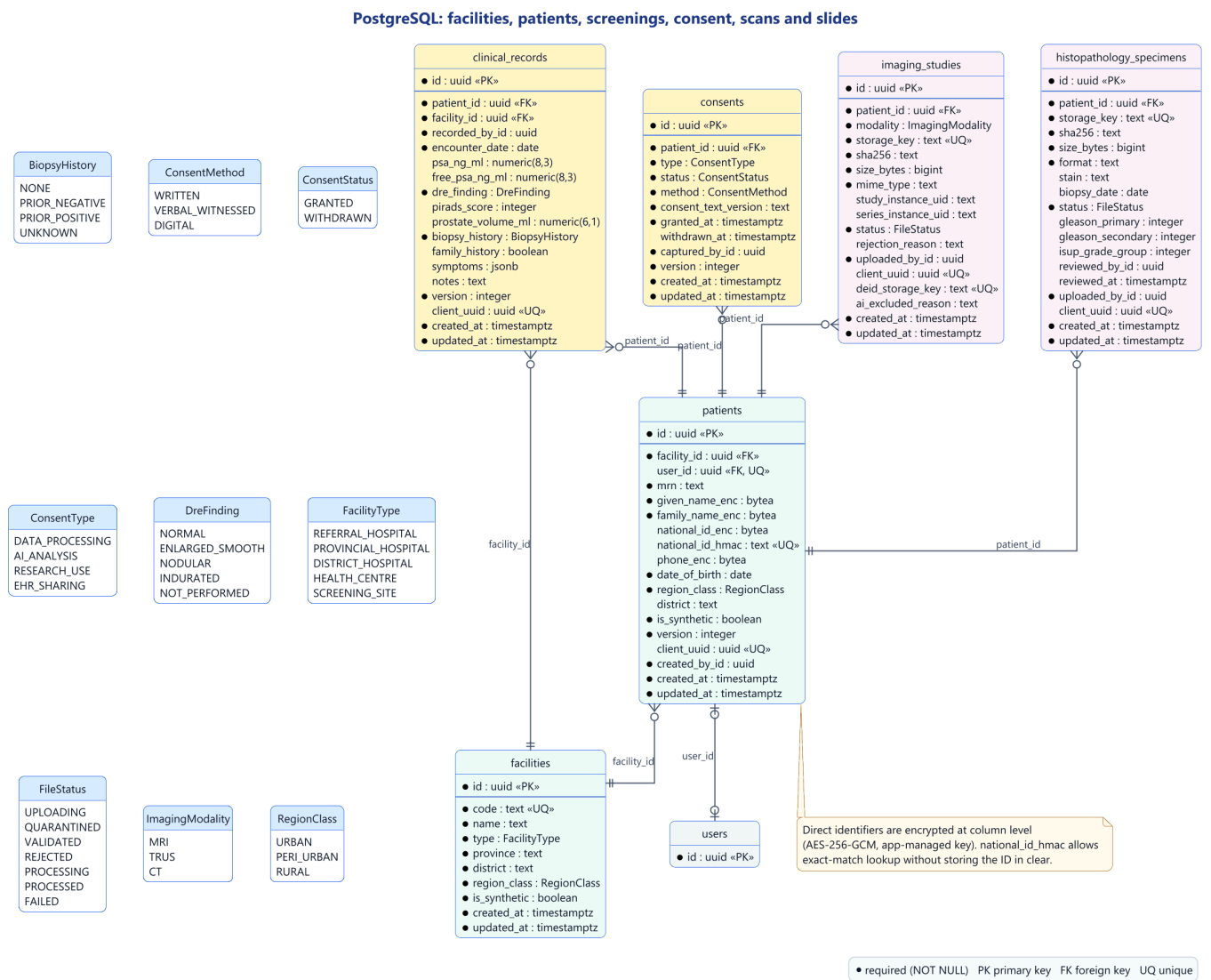
Figure 3.30: ERD: accounts, roles and sessions

PostgreSQL: accounts, roles and sessions



Files: [PNG](#) · [SVG](#) · source generated: [erd-accounts.puml](#)

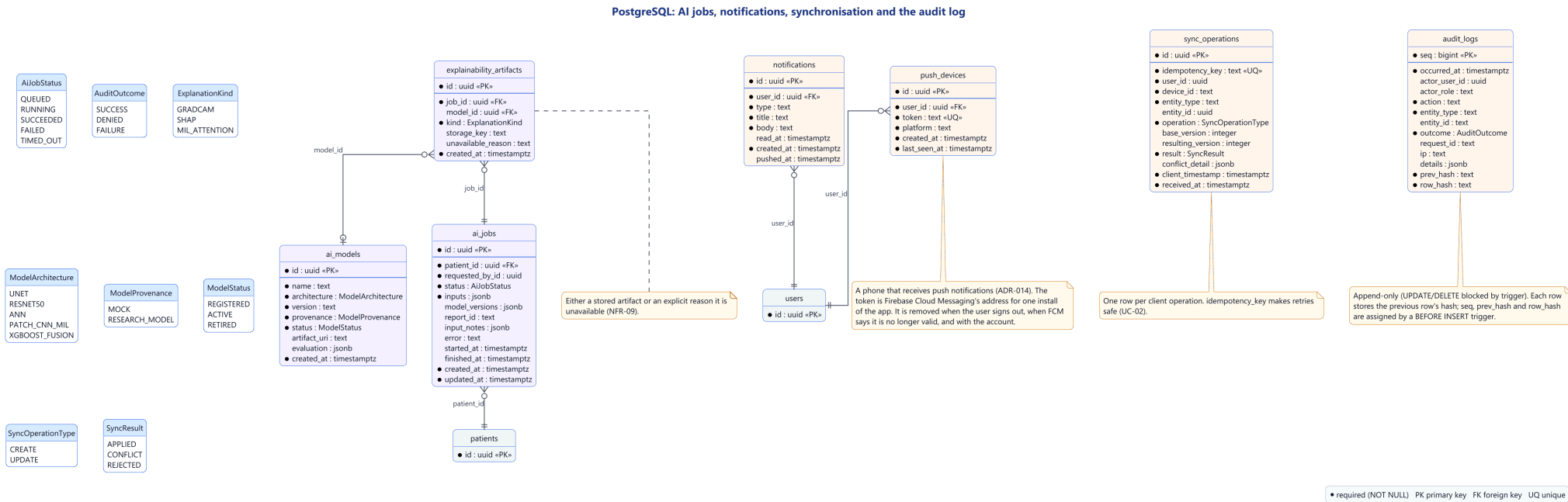
Figure 3.31: ERD: facilities, patients, screenings, consent, scans and slides



Files: PNG · SVG · source generated: erd-patients.puml

Figure 3.32: ERD: AI jobs, notifications, synchronisation and the audit log

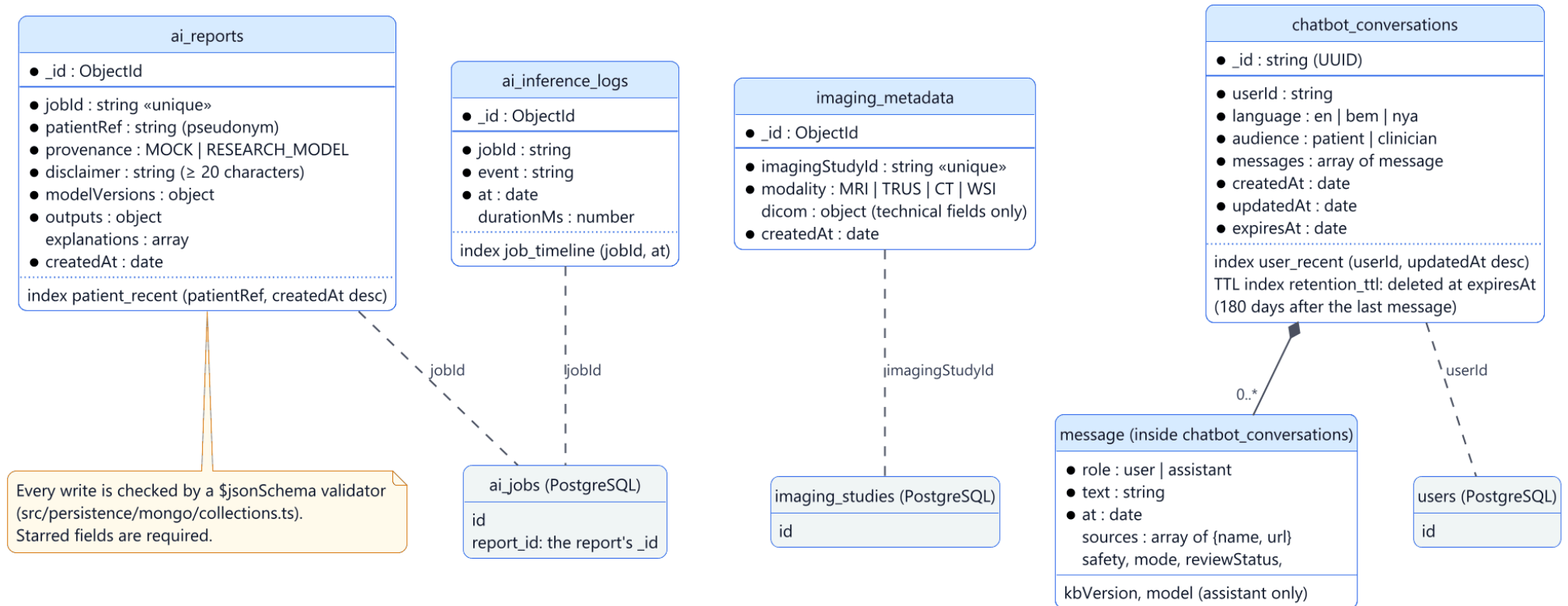
Also the phones that receive push notifications (`push_devices` , ADR-014), and when each notification was pushed (`notifications.pushed_at`).



Files: [PNG](#) · [SVG](#) · source generated: [erd-ai-sync-audit.puml](#)

Figure 3.33: MongoDB: the four document collections and their rules

MongoDB 8.3: the four document collections and their rules (pca_mhealth)

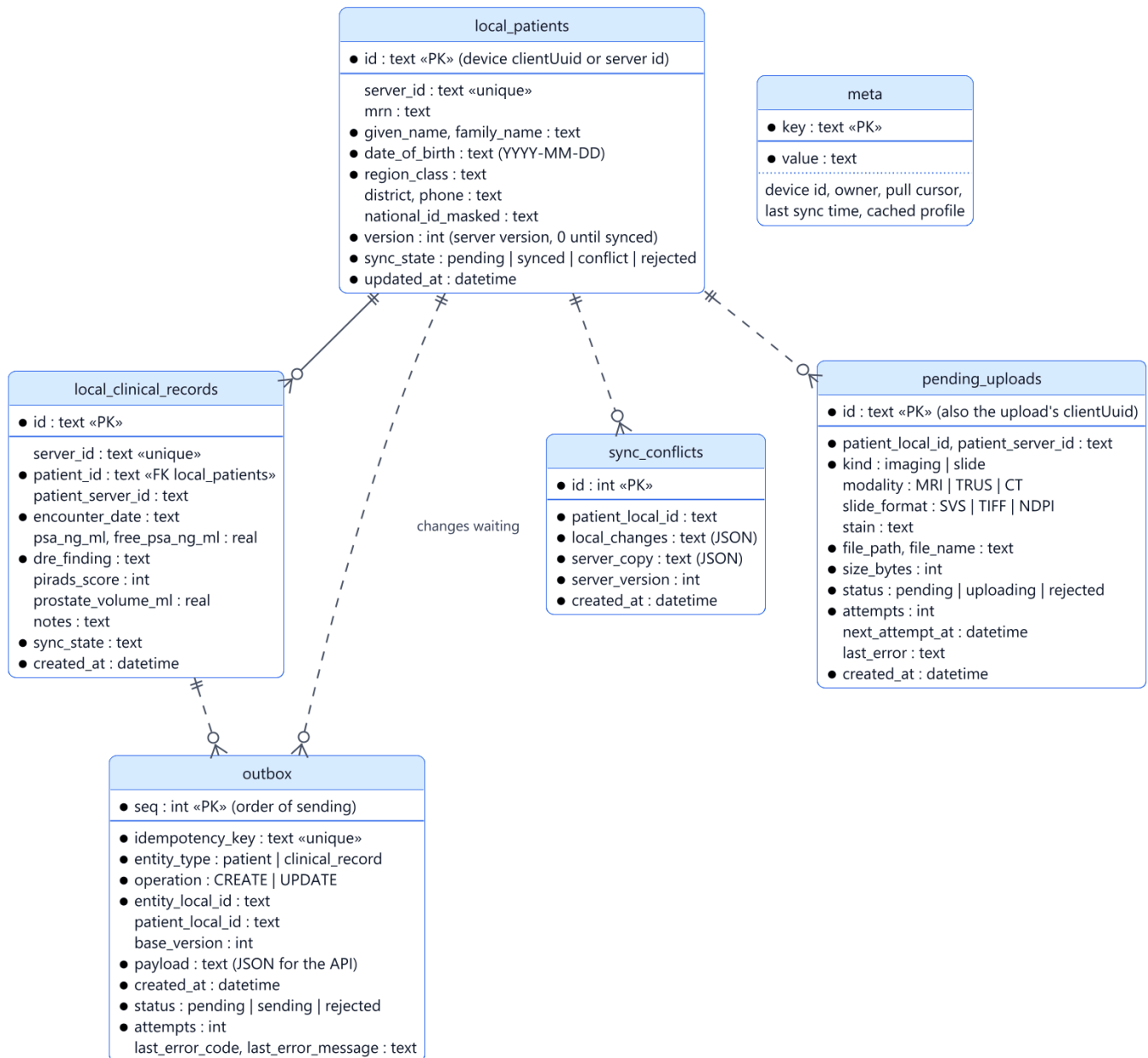


Files: [PNG](#) · [SVG](#) · source [database-mongodb.puml](#)

Figure 3.34: The phone's encrypted database

The six tables the app keeps on the phone (Drift with SQLCipher) for offline work.

The phone's encrypted database (Drift + SQLCipher, AES-256; key in the Android Keystore)



Files: [PNG](#) · [SVG](#) · source [database-phone.puml](#)

Facts for the normalisation section

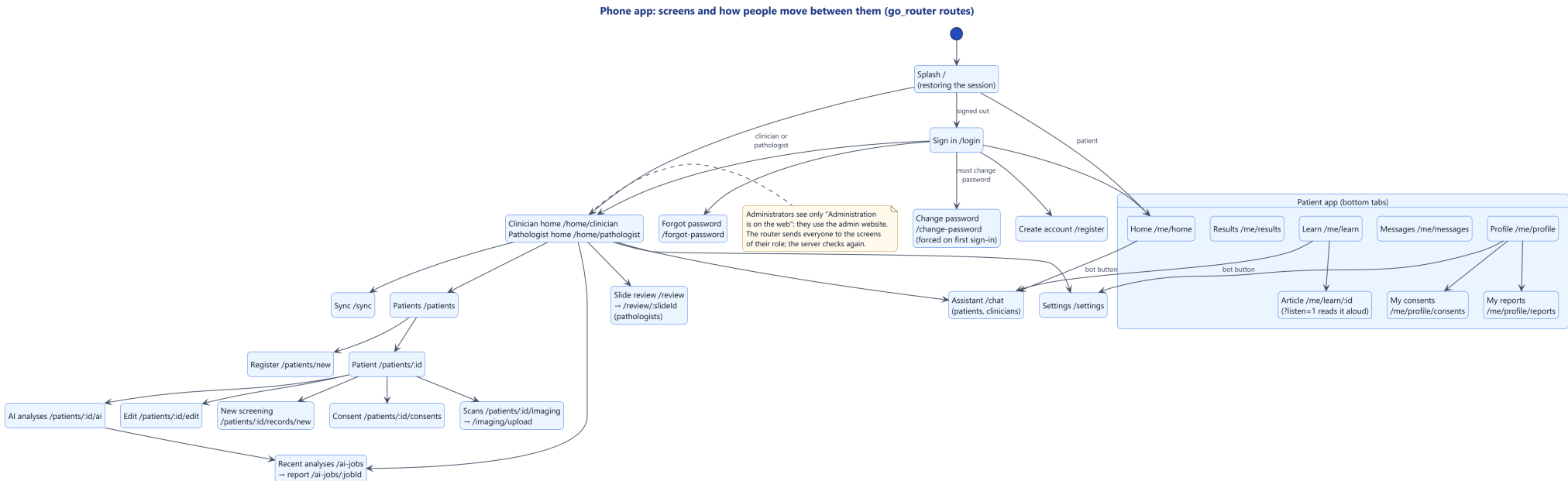
These are facts about the schema you can use when you discuss normalisation:

- **Keys:** every table has a primary key: a UUID `id`, except the two link tables `user_roles` (`user_id`, `role_id`) and `role_permissions` (`role_id`, `permission_id`), which have composite keys, and `audit_logs`, keyed by a sequence number `seq`.
- **Alternate keys** (unique columns): for example `users.email`, and `push_devices.token`, so one phone is registered only once.

- **Many-to-many relations** (users and roles, roles and permissions) are resolved by those two link tables.
- **Second normal form:** the only composite keys are those of the link tables; `role_permissions` has no other column, and `user_roles.assigned_at` depends on the whole key.
- **Third normal form:** facility details are stored once in `facilities` and referenced by `facility_id`; role names once in `roles`. PSA density and the free-to-total PSA ratio are calculated when a record is read, not stored.
- **Deliberate exceptions:**
 - Some columns hold a JSON value: `clinical_records.symptoms`, `ai_jobs.inputs`, `model_versions` and `input_notes`, `ai_models.evaluation`, `sync_operations.conflict_detail`, `audit_logs.details`.
 - `histopathology_specimens.isup_grade_group` is computed by the server from the two Gleason values and stored with the review.
 - AI reports, chats and scan details live in MongoDB as documents, linked by id.
- **Integrity rules in the database:** 18 foreign keys, unique keys, CHECK constraints on clinical values (PSA, PI-RADS, Gleason, ISUP), and triggers that make `audit_logs` append-only and chain each row's hash to the previous one.

Figure 3.35: The app's screens and navigation

Every route of the app and how people move between screens, by role.



Files: [PNG](#) · [SVG](#) · source [navigation-app.puml](#)

Figure 3.36: Interface design: patient screens

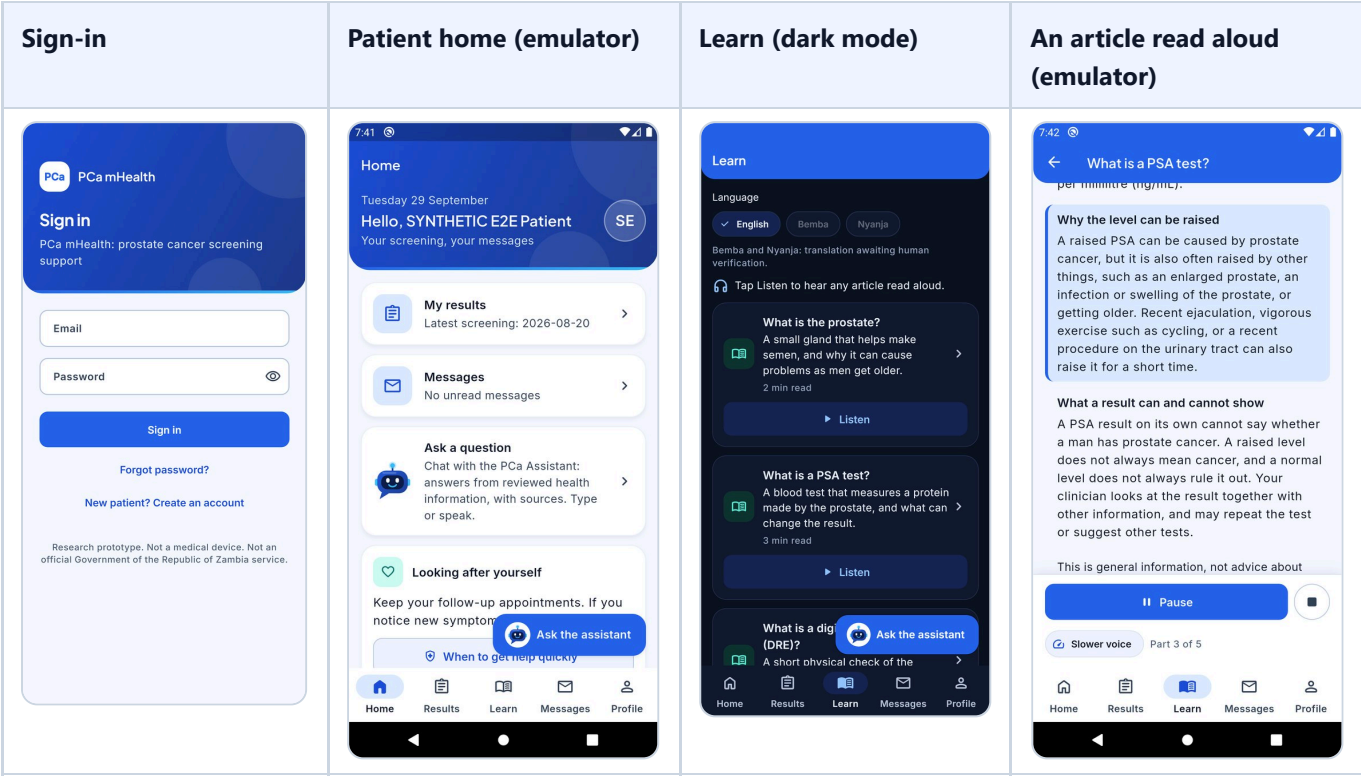
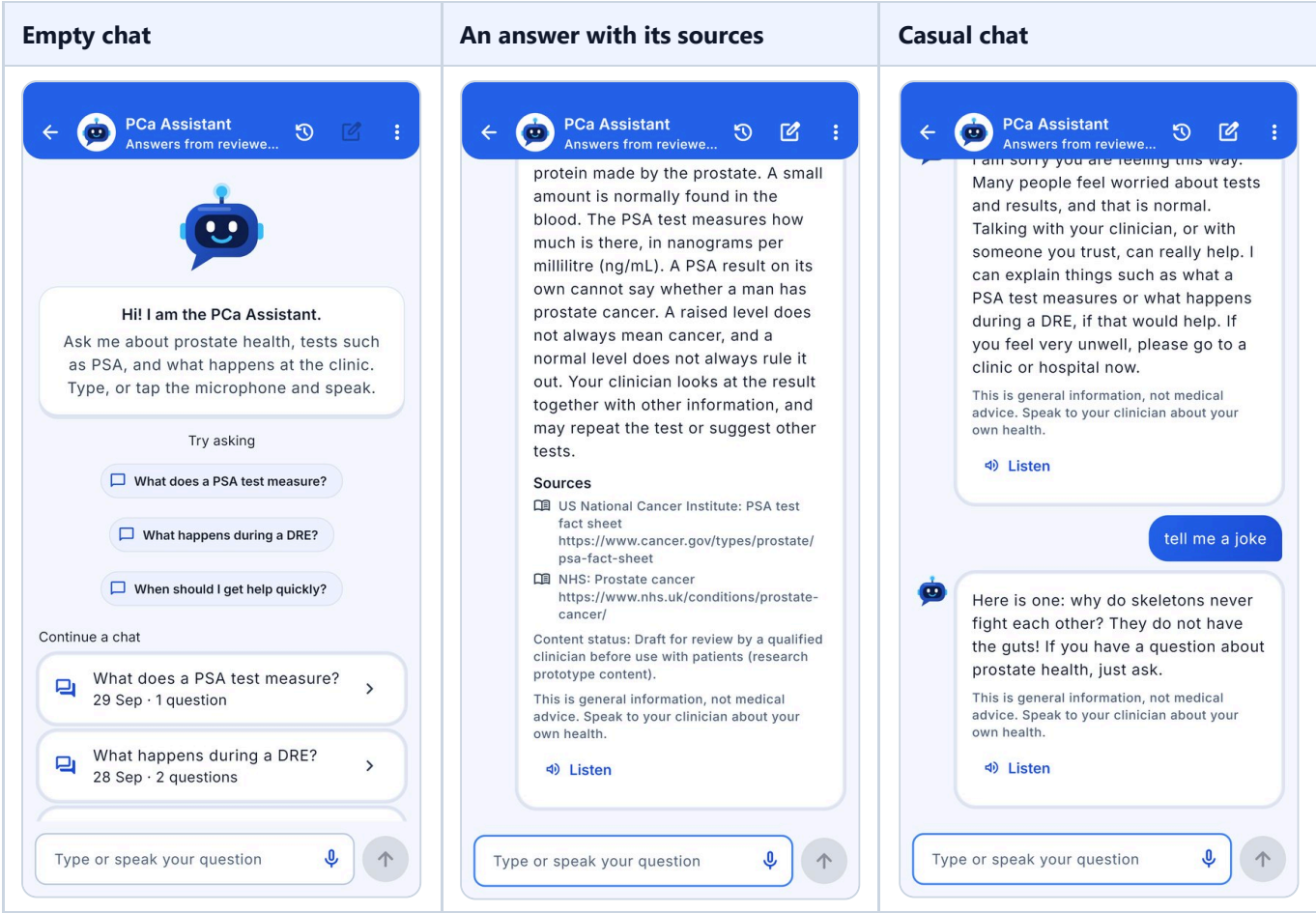
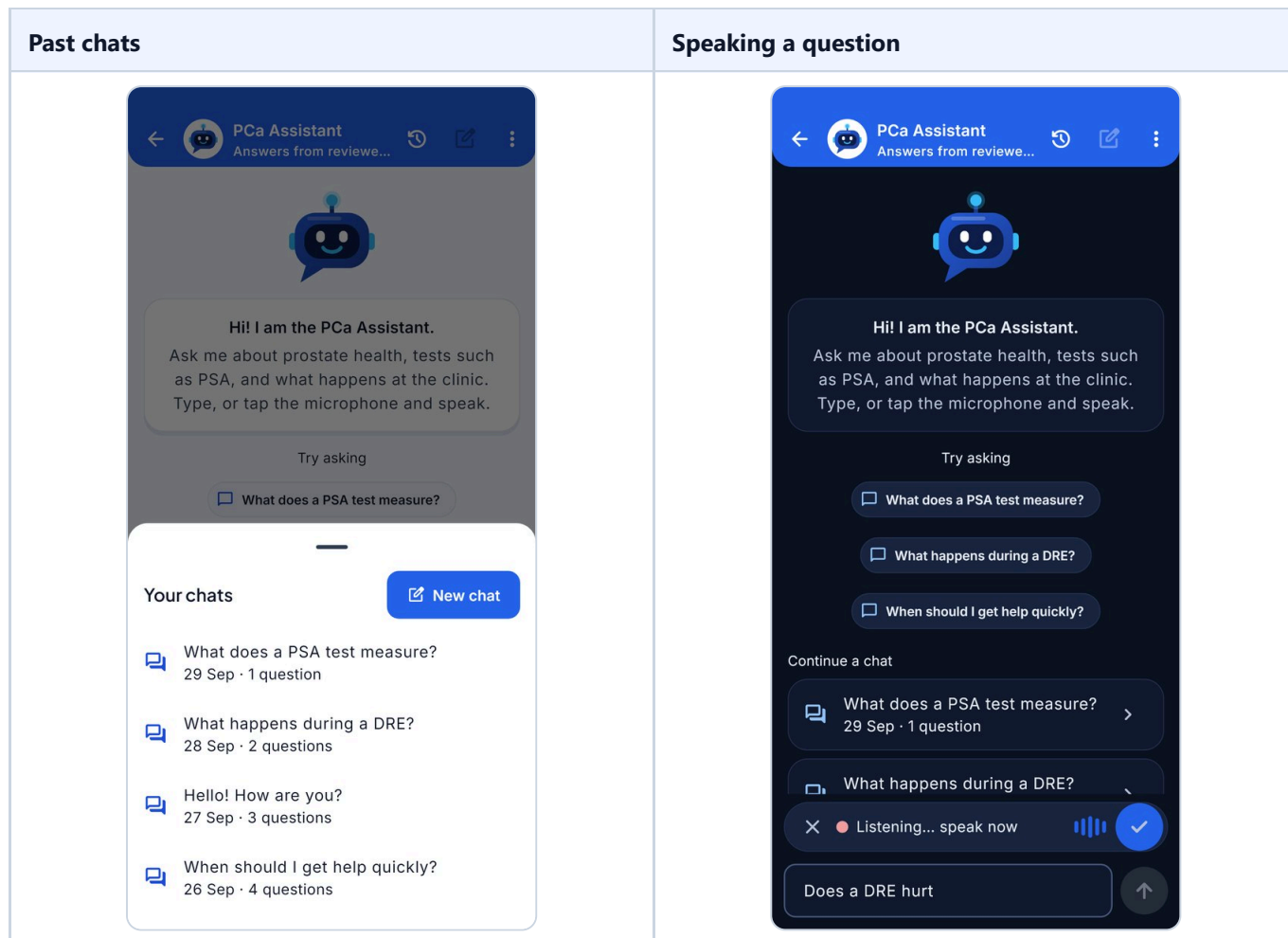


Figure 3.37: Interface design: the assistant





The answer is the AI service's real answer to that question; the casual replies are the backend's real replies; in the recording picture a stand-in plays the part of the phone's speech service.

Figure 3.38: Interface design: clinician home

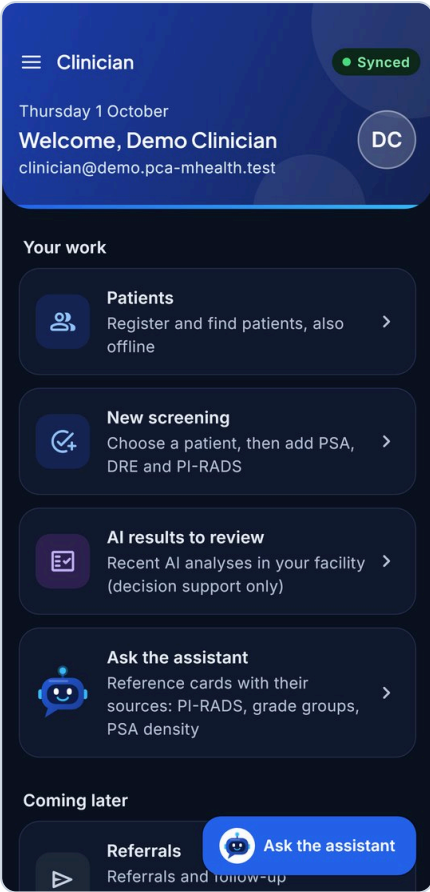
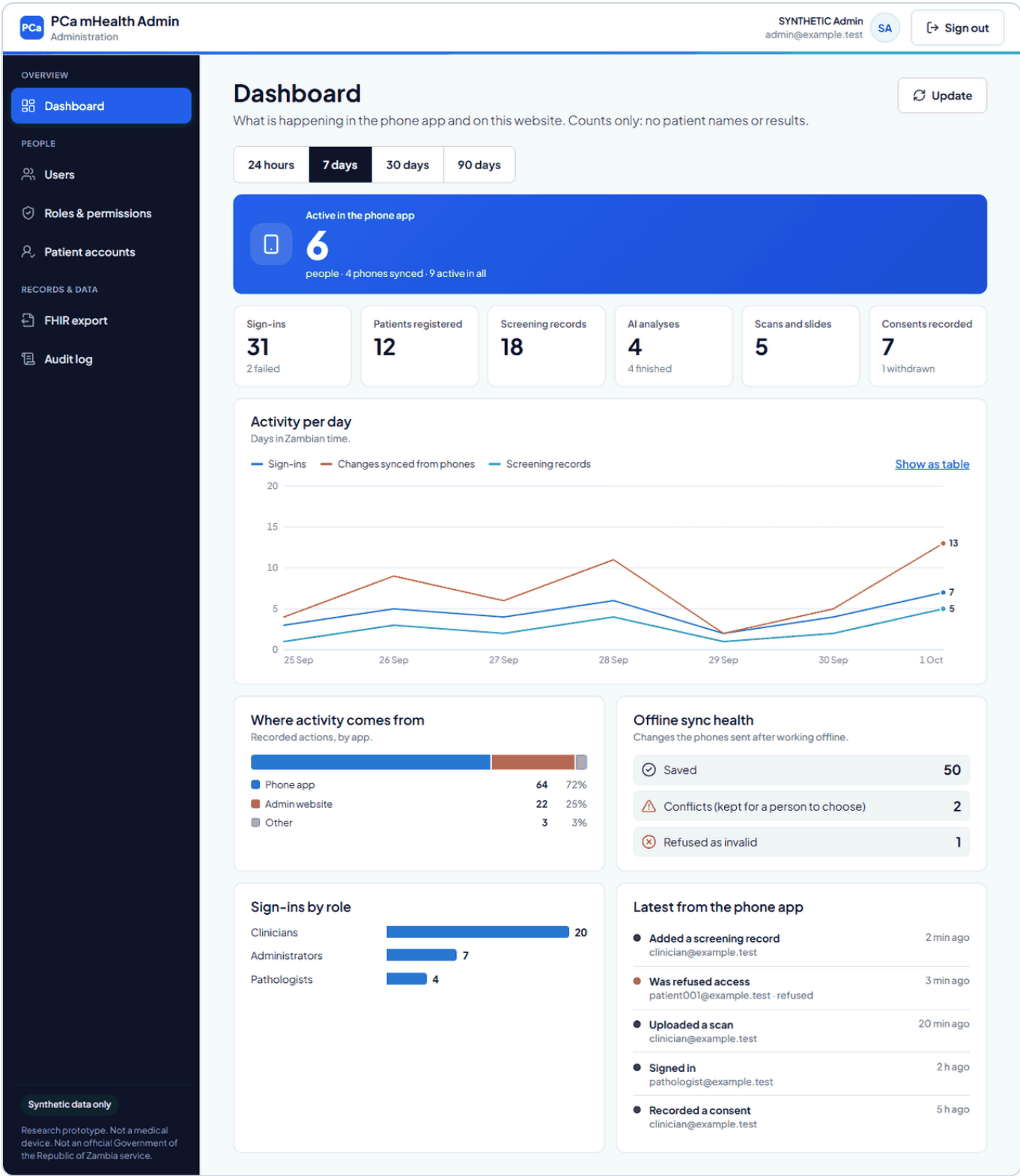


Figure 3.39: Interface design: the admin website

The dashboard, the accounts list and the delete dialog. Synthetic API answers were used for these pictures; the dashboard's numbers are made up for the picture.



PCa

PCa mHealth Admin

Administration

SYNTHETIC Admin

admin@example.test

SA

Sign out

OVERVIEW

Dashboard

PEOPLE

Users

Roles & permissions

Patient accounts

RECORDS & DATA

FHIR export

Audit log

Synthetic data only

Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.

Users

Everyone who can sign in. Open an account to change its roles, facility or status.

Search by name or email

All roles

Search

NAME	EMAIL	ROLES	STATUS
SYNTHETIC Admin	admin@example.test	Administrator	ACTIVE
SYNTHETIC Clinician	clinician@example.test	Clinician	ACTIVE
SYNTHETIC Pathologist	pathologist@example.test	Pathologist / Radiologist	ACTIVE
SYNTHETIC Patient 001	patient001@example.test	Patient	ACTIVE
SYNTHETIC Patient 002	patient002@example.test	Patient	LOCKED
SYNTHETIC Test account	e2e-patient@example.test	Patient	ACTIVE
SYNTHETIC Former staff	former@example.test	Clinician	DISABLED

Add staff user

PCa

PCa mHealth Admin

Administration

SYNTHETIC Admin

admin@example.test

SA

Sign out

OVERVIEW

Dashboard

PEOPLE

Users

Roles & permissions

Patient accounts

RECORDS & DATA

FHIR export

Audit log

Synthetic data only

Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.

Users

SYNTHETIC Test account

ACTIVE

e2e-patient@example.test

Last sign-in 9/29/2026, 10:00:00 AM

Roles

What this p

☒ Patient

☐ Clinician

☐ Pathologist

☐ Administrator

Delete this account?

e2e-patient@example.test will be deleted with their roles, sessions, messages and assistant chats. This cannot be undone. A linked patient record stays; only the sign-in goes.

Accounts that registered patients, recorded screenings or consents, uploaded scans or asked for AI analyses cannot be deleted, so the medical history stays traceable. Switch those off instead.

CancelDelete account

Save changes

Account actions

A reset gives a one-time password and signs the person out everywhere. Delete removes an account that has no clinical history; other accounts can be switched off with Account active.

Reset password

Delete account

Chapter 4: System implementation and testing

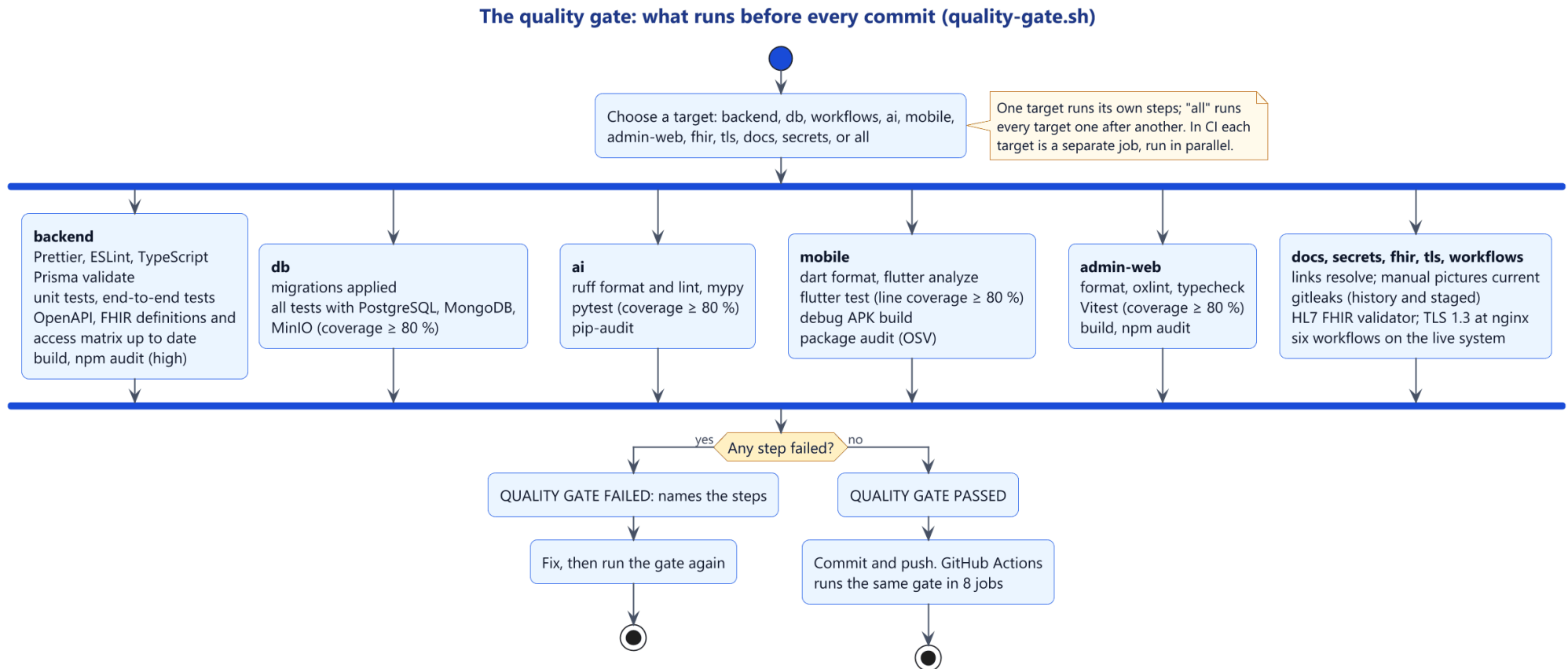
4.1 System guide and 4.2 Installation manual

The [operations manual](#) is the system guide and installation manual: what is installed, how to start, check and stop everything (§2 to §4), the phone app (§10) and the admin website (§11). [how-to-test.md](#) walks through every feature by hand. Figure 3.22 (deployment) shows where each part runs.

4.3 Testing plan and test output

Figure 4.1: The quality gate

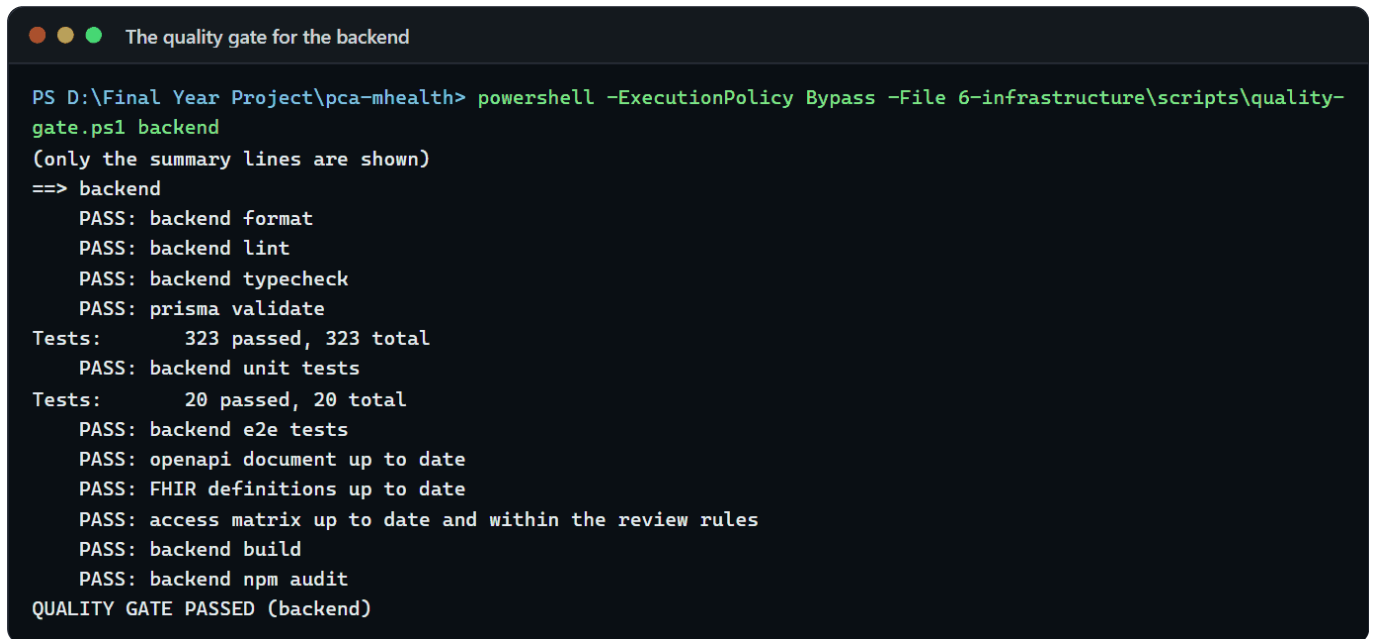
What runs before every commit and in CI: formatting, lint, types, tests with at least 80 % coverage, builds and security audits, for each part.



Files: [PNG](#) · [SVG](#) · source [activity-quality-gate.puml](#)

Figure 4.2: Test output: the backend gate

The real output of the backend gate (summary lines). The testing strategy is in [testing-strategy.md](#); the counts for every part are in §12 of the operations manual.



```
PS D:\Final Year Project\pca-mhealth> powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend
(only the summary lines are shown)
==> backend
    PASS: backend format
    PASS: backend lint
    PASS: backend typecheck
    PASS: prisma validate
Tests:      323 passed, 323 total
    PASS: backend unit tests
Tests:      20 passed, 20 total
    PASS: backend e2e tests
    PASS: openapi document up to date
    PASS: FHIR definitions up to date
    PASS: access matrix up to date and within the review rules
    PASS: backend build
    PASS: backend npm audit
QUALITY GATE PASSED (backend)
```

The user acceptance test (proposal §3.8.2) has its own kit in [uat/](#): the plan, the information sheet and consent form, task sheets for each role, the SUS questionnaire and a results template whose tables fit this section. It may only be used after ethics approval.

4.4 Main function codes

The code that meets each objective of the proposal. Every picture shows the real file with its line numbers.

Objective 1: a secure, offline-first mobile application (AES-256, TLS 1.3, RBAC, FHIR R4)

Figure 4.3: The phone's encrypted database (SQLCipher key in the Keystore)

```
1-presentation-layer/mobile-app/lib/core/db/open_database.dart lines 16-57

16 /// Opens the on-device database, encrypted with SQLCipher (AES-256, NFR-01).
17 /// The 256-bit key is random per installation and kept only in the Android
18 /// Keystore / iOS Keychain. If the key is lost (e.g. app data restored without
19 /// the Keystore), the unreadable file is replaced by an empty database; the
20 /// data can be downloaded again from the server.
21 Future<AppDatabase> openAppDatabase({FlutterSecureStorage? storage}) async {
22   final secure = storage ?? const FlutterSecureStorage();
23   final dir = await getApplicationSupportDirectory();
24   final file = File(p.join(dir.path, _fileName));
25
26   var key = await secure.read(key: _keyName);
27   if (key == null) {
28     if (await file.exists()) await file.delete();
29     key = _randomHexKey();
30     await secure.write(key: _keyName, value: key);
31   }
32   final hexKey = key;
33
34   return AppDatabase(
35     NativeDatabase.createInBackground(
36       file,
37       isolateSetup: useSqlCipher,
38       setup: (db) {
39         // Fails fast if plain SQLite was loaded instead of SQLCipher.
40         final version = db.select('PRAGMA cipher_version;');
41         if (version.isEmpty) {
42           throw StateError(
43             'SQLCipher is not available; refusing to store data unencrypted',
44           );
45         }
46         db.execute("PRAGMA key = \"x'$hexKey'\";");
47         // Reading the schema verifies the key.
48         db.select('SELECT count(*) FROM sqlite_master;');
49       },
50     ),
51   );
52 }
53
54 /// Loads SQLCipher instead of the system SQLite on Android.
55 void useSqlCipher() {
56   open.overrideFor(OperatingSystem.android, openCipherOnAndroid);
57 }
```

Figure 4.4: The phone's sync engine

```
1-presentation-layer/mobile-app/lib/core/sync/sync_engine.dart lines 85-130

85 Future<SyncOutcome> _run() async {
86   _emit(SyncPhase.syncing);
87   if (!_recovered) {
88     await store.recoverInterrupted();
89     _recovered = true;
90   }
91   List<OutboxData> inFlight = const [];
92   try {
93     final deviceId = await store.deviceId();
94     for (var i = 0; i < _maxBatchesPerRun; i++) {
95       inFlight = await store.takeBatch(batchSize);
96       if (inFlight.isEmpty) break;
97       await _push(deviceId, inFlight);
98       inFlight = const [];
99     }
100    await _pull();
101
102    final finished = _now();
103    _failures = 0;
104    _nextAttemptAt = null;
105    await store.setMeta(
106      MetaKey.lastSyncAt,
107      finished.toUtc().toIso8601String(),
108    );
109    _emit(SyncPhase.idle, lastSyncAt: finished);
110    return SyncOutcome.completed;
111  } catch (e) {
112    if (inFlight.isNotEmpty) {
113      await store.releaseBatch(inFlight.map((o) => o.seq));
114    }
115    _failures++;
116    var wait = backoffFor(_failures);
117    // A rate-limited server says how long to wait; never retry sooner.
118    final serverWait = e is ApiException ? e.retryAfter : null;
119    if (serverWait != null && serverWait > wait) wait = serverWait;
120    _nextAttemptAt = _now().add(wait);
121    final offline = e is ApiException && e.isNetwork;
122    _emit(
123      offline ? SyncPhase.offline : SyncPhase.failed,
124      error: e is ApiException
125        ? e.message
126        : 'Sync failed. It will try again.',
127    );
128    return offline ? SyncOutcome.offline : SyncOutcome.failed;
129  }
130 }
```


Figure 4.5: The server applies each synced change once (idempotency)

```
3-application-logic/backend/src/services/sync/sync.service.ts lines 124-196

124 private async apply(
125   op: SyncOperationDto,
126   deviceId: string,
127   failedPatients: ReadonlySet<string>,
128   user: AuthenticatedUser,
129   ctx: RequestContext,
130 ): Promise<SyncOperationResult> {
131   const stored = await this.prisma.syncOperation.findUnique({
132     where: { idempotencyKey: op.idempotencyKey },
133   });
134   if (stored) return this.replay(stored, op, user, ctx);
135
136   let outcome: Omit<SyncOperationResult, 'idempotencyKey' | 'replayed'>;
137   try {
138     if (op.patientId && failedPatients.has(op.patientId)) {
139       // Not stored: once the patient is fixed and synced, this can apply.
140       return {
141         idempotencyKey: op.idempotencyKey,
142         replayed: false,
143         result: 'REJECTED',
144         entityType: op.entityType,
145         error: {
146           code: 'DEPENDENCY_FAILED',
147           message:
148             'The patient for this record could not be saved on the server',
149         },
150       };
151     }
152     outcome = await this.execute(op, user, ctx);
153   } catch (e) {
154     if (!(e instanceof Rejected)) throw e;
155     outcome = {
156       result: 'REJECTED',
157       entityType: op.entityType,
158       error: e.error,
159     };
160   }
161
162   try {
163     await this.prisma.syncOperation.create({
164       data: {
165         idempotencyKey: op.idempotencyKey,
166         userId: user.id,
167         deviceId,
168         entityType: op.entityType,
169         entityId: outcome.entityId ?? null,
170         operation: op.operation,
171         baseVersion: op.baseVersion ?? null,
172         resultingVersion: outcome.version ?? null,
173         result: outcome.result,
174         conflictDetail: outcome.error
175           ? (JSON.parse(
176               JSON.stringify(outcome.error),
177             ) as Prisma.InputJsonObject)
178           : Prisma.JsonNull,
179         clientTimestamp: new Date(op.clientTimestamp),
180       },
181     });
182   } catch (e) {
183     // Two concurrent batches with the same key: the first one wins.
184     if (
185       e instanceof Prisma.PrismaClientKnownRequestError &&
186       e.code === 'P2002'
187     ) {
188       const winner = await this.prisma.syncOperation.findUniqueOrThrow({
189         where: { idempotencyKey: op.idempotencyKey },
190       });
191       return this.replay(winner, op, user, ctx);
192     }
193     throw e;
194   }
195   return { idempotencyKey: op.idempotencyKey, replayed: false, ...outcome };
196 }
```

Figure 4.6: AES-256-GCM encryption of identifiers, and HMAC look-ups

```
3-application-logic/backend/src/persistence/crypto/field-crypto.ts lines 21-82

21 export class FieldCrypto {
22   constructor(
23     private readonly encryptionKey: Buffer,
24     private readonly hmacKey: Buffer,
25   ) {
26     if (encryptionKey.length !== KEY_LENGTH || hmacKey.length !== KEY_LENGTH) {
27       throw new Error('Field encryption and HMAC keys must each be 32 bytes');
28     }
29   }
30
31   static fromEnv(env: Record<string, string | undefined>): FieldCrypto {
32     const enc = env.FIELD_ENCRYPTION_KEY_BASE64;
33     const mac = env.FIELD_HMAC_KEY_BASE64;
34     if (!enc) throw new Error('FIELD_ENCRYPTION_KEY_BASE64 is not set');
35     if (!mac) throw new Error('FIELD_HMAC_KEY_BASE64 is not set');
36     return new FieldCrypto(
37       Buffer.from(enc, 'base64'),
38       Buffer.from(mac, 'base64'),
39     );
40   }
41
42   /** Returns a standalone Uint8Array, the type Prisma uses for `Bytes` columns. */
43   encrypt(plaintext: string): Uint8Array<ArrayBuffer> {
44     const iv = randomBytes(IV_LENGTH);
45     const cipher = createCipheriv('aes-256-gcm', this.encryptionKey, iv);
46     const body = Buffer.concat([
47       cipher.update(plaintext, 'utf8'),
48       cipher.final(),
49     ]);
50     return new Uint8Array(
51       Buffer.concat([Buffer.of(FORMAT_VERSION), iv, cipher.getAuthTag(), body]),
52     );
53   }
54
55   decrypt(value: Uint8Array): string {
56     const sealed = Buffer.from(
57       value.buffer,
58       value.byteOffset,
59       value.byteLength,
60     );
61     if (
62       sealed.length < 1 + IV_LENGTH + TAG_LENGTH ||
63       sealed[0] !== FORMAT_VERSION
64     ) {
65       throw new Error('Unsupported encrypted field version or truncated value');
66     }
67     const iv = sealed.subarray(1, 1 + IV_LENGTH);
68     const tag = sealed.subarray(1 + IV_LENGTH, 1 + IV_LENGTH + TAG_LENGTH);
69     const body = sealed.subarray(1 + IV_LENGTH + TAG_LENGTH);
70     const decipher = createDecipheriv('aes-256-gcm', this.encryptionKey, iv);
71     decipher.setAuthTag(tag);
72     return Buffer.concat([decipher.update(body), decipher.final()]).toString(
73       'utf8',
74     );
75   }
76
77   hmac(value: string): string {
78     return createHmac('sha256', this.hmacKey)
79       .update(value, 'utf8')
80       .digest('hex');
81   }
82 }
```

Figure 4.7: Sign-in with lockout and session tokens

```
3-application-logic/backend/src/services/auth/auth.service.ts lines 189-253

189 async login(
190   emailInput: string,
191   password: string,
192   ctx: RequestContext,
193 ): Promise<LoginResult> {
194   const email = normalizeEmail(emailInput);
195   const user = await this.prisma.user.findUnique({
196     where: { email },
197     include: USER_WITH_ACCESS,
198   });
199
200   if (!user) {
201     await this.passwords.verifyDummy(password);
202     await this.recordLogin(null, 'FAILURE', 'unknown_account', ctx);
203     throw invalidCredentials();
204   }
205   if (user.status === 'DISABLED') {
206     await this.passwords.verifyDummy(password);
207     await this.recordLogin(user, 'FAILURE', 'disabled', ctx);
208     throw invalidCredentials();
209   }
210   if (user.lockedUntil && user.lockedUntil > new Date()) {
211     await this.recordLogin(user, 'DENIED', 'locked', ctx);
212     throw accountLocked();
213   }
214
215   if (!(await this.passwords.verify(user.passwordHash, password))) {
216     await this.registerFailedAttempt(user, ctx);
217     throw invalidCredentials();
218   }
219
220   const familyId = randomUUID();
221   const refresh = this.tokens.newRefreshToken();
222   await this.prisma.$transaction(async (tx) => {
223     await tx.user.update({
224       where: { id: user.id },
225       data: {
226         failedLoginCount: 0,
227         lockedUntil: null,
228         lastLoginAt: new Date(),
229       },
230     });
231     await tx.refreshToken.create({
232       data: {
233         userId: user.id,
234         familyId,
235         tokenHash: refresh.hash,
236         expiresAt: this.tokens.refreshExpiry(),
237       },
238     });
239     await this.recordLogin(user, 'SUCCESS', undefined, ctx, tx);
240   });
241
242   const roles = rolesOf(user);
243   return {
244     ...(await this.sessionTokens(user.id, familyId, refresh.token)),
245     user: {
246       id: user.id,
247       email: user.email,
248       displayName: user.displayName,
249       roles,
250       mustChangePassword: user.mustChangePassword,
251     },
252   };
253 }
```

Figure 4.8: Role-based access control: the permissions guard

```
3-application-logic/backend/src/gateway/access/permissions.guard.ts lines 29–44

29  async canActivate(context: ExecutionContext): Promise<boolean> {
30    const targets = [context.getHandler(), context.getClass()];
31    if (this.reflector.getAllAndOverride<boolean>(IS_PUBLIC, targets))
32      return true;
33
34    const required =
35      this.reflector.getAllAndMerge<PermissionCode[]>(
36        REQUIRED_PERMISSIONS,
37        targets,
38      ) ?? [];
39    if (required.length === 0) return true;
40
41    const req = context.switchToHttp().getRequest<AuthenticatedRequest>();
42    const user = req.user;
43    const missing = required.filter((p) => !user?.permissions.includes(p));
44    if (missing.length === 0) return true;
```

Figure 4.9: TLS 1.3 only at the reverse proxy

```
2-api-gateway/reverse-proxy/nginx.conf lines 33–66

33  server {
34    listen 443 ssl;
35    http2 on;
36    server_name api.example.org;
37
38    ssl_certificate      /etc/nginx/certs/fullchain.pem;
39    ssl_certificate_key  /etc/nginx/certs/privkey.pem;
40    ssl_protocols TLSv1.3;
41    ssl_prefer_server_ciphers off;
42    ssl_session_cache shared:SSL:10m;
43    ssl_session_tickets off;
44
45    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
46    add_header X-Content-Type-Options nosniff always;
47
48    # Imaging and slide uploads stream straight through to the backend.
49    client_max_body_size 2100m;
50    proxy_request_buffering off;
51    proxy_read_timeout 120s;
52
53    location /api/ {
54      proxy_pass http://pca_api;
55      proxy_http_version 1.1;
56      proxy_set_header Connection "";
57      proxy_set_header Host $host;
58      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
59      proxy_set_header X-Forwarded-Proto https;
60    }
61
62    # Everything else is refused: the API is the only door.
63    location / {
64      return 404;
65    }
66  }
```

Figure 4.10: Safe Harbor de-identification rules for the FHIR export

```
3-application-logic/backend/src/services/fhir/deidentify.ts lines 17-46

17 export const SAFE_HARBOR: readonly SafeHarborRule[] = [
18   {
19     n: 1,
20     identifier: 'Names',
21     inThisSystem: "Patient's given and family name (stored encrypted)",
22     inTheExport: 'Removed. Names are never decrypted for an export.',
23   },
24   {
25     n: 2,
26     identifier: 'Geographic subdivisions smaller than a state',
27     inThisSystem:
28       "Patient's district; the facility's name, code, district and province",
29     inTheExport:
30       'Removed. Only the country (ZM) and the urban / peri-urban / rural class are kept.',
31   },
32   {
33     n: 3,
34     identifier:
35       'All elements of dates (except year) directly related to a person; ages over 89',
36     inThisSystem:
37       'Date of birth, screening dates, biopsy dates, review, consent and AI dates',
38     inTheExport:
39       'Year only. From age 90 the birth year is removed too and the age is given as "90 or older".',
40   },
41   {
42     n: 4,
43     identifier: 'Telephone numbers',
44     inThisSystem: "Patient's phone number (stored encrypted)",
45     inTheExport: 'Removed.',
46   },
47 ]
```

Figure 4.11: Push notifications: the outbox sends each new notification once (ADR-014)

```
3-application-logic/backend/src/services/notifications/push/push-outbox.service.ts lines 84-145

84 private async run(): Promise<PushRun> {
85   const result: PushRun = {
86     notifications: 0,
87     sent: 0,
88     failed: 0,
89     removedDevices: 0,
90   };
91   if (!this.sender.enabled) return result;
92   const claimed = await this.prisma.$queryRaw<Claimed[]>`
93     UPDATE notifications SET pushed_at = now()
94     WHERE id IN (
95       SELECT id FROM notifications
96       WHERE pushed_at IS NULL
97       AND created_at > now() - make_interval(mins => ${this.config.push.maxAgeMinutes}::int)
98       ORDER BY created_at
99       LIMIT ${BATCH}
100     FOR UPDATE SKIP LOCKED)
101     RETURNING id, user_id AS "userId", type, title, body`;
102   result.notifications = claimed.length;
103   if (claimed.length === 0) return result;
104
105   // Disabled accounts get nothing; a locked one (wrong passwords) still does.
106   const devices = await this.prisma.pushDevice.findMany({
107     where: {
108       userId: { in: [...new Set(claimed.map((n) => n.userId))] },
109       user: { status: { not: 'DISABLED' } },
110     },
111     select: { id: true, userId: true, token: true },
112   });
113   const invalid = new Set<string>();
114   for (const notification of claimed) {
115     for (const device of devices) {
116       if (device.userId !== notification.userId || invalid.has(device.id)) {
117         continue;
118       }
119       const outcome = await this.sender.send({
120         token: device.token,
121         title: notification.title,
122         body: notification.body,
123         data: { notificationId: notification.id, type: notification.type },
124       });
125       if (outcome === 'sent') {
126         result.sent++;
127       } else {
128         result.failed++;
129         if (outcome === 'invalid-token') invalid.add(device.id);
130       }
131     }
132   }
133   if (invalid.size > 0) {
134     const { count } = await this.prisma.pushDevice.deleteMany({
135       where: { id: { in: [...invalid] } },
136     });
137     result.removedDevices = count;
138   }
139   if (result.failed > 0) {
140     this.logger.warn(
141       `Push: ${result.sent} sent, ${result.failed} not sent; ${result.removedDevices} phones no longer
142       known to Firebase were removed`,
143     );
144   }
145   return result;
146 }
```

Objective 2: the multi-modal AI inference engine (the pipeline is built; the trained models are not, so the modules are labelled mocks)

Figure 4.12: Requesting an analysis: consent first

```
3-application-logic/backend/src/services/ai/ai.service.ts lines 204–242

204  async request(
205    patientId: string,
206    user: AuthenticatedUser,
207    ctx: RequestContext,
208  ): Promise<AiJobView> {
209    if (!this.broker.enabled) {
210      throw new ServiceUnavailableException({
211        code: 'AI_UNAVAILABLE',
212        message: 'AI analysis is switched off on this server',
213      });
214    }
215    const patient = await this.patients.requireInFacility(user, patientId);
216    if (!(await this.clinical.hasActiveConsent(patient.id, 'AI_ANALYSIS'))) {
217      throw new ConflictException({
218        code: 'CONSENT_REQUIRED',
219        message:
220          'The patient has not consented to AI analysis. Record their consent first.',
221      });
222    }
223    const inProgress = await this.prisma.aiJob.findFirst({
224      where: { patientId: patient.id, status: { in: ['QUEUED', 'RUNNING'] } },
225    });
226    if (inProgress) {
227      throw new ConflictException({
228        code: 'AI_JOB_IN_PROGRESS',
229        message: 'An analysis for this patient is already running',
230        details: { jobId: inProgress.id },
231      });
232    }
233    const record = await this.prisma.clinicalRecord.findFirst({
234      where: { patientId: patient.id },
235      orderBy: [{ encounterDate: 'desc' }, { createdAt: 'desc' }],
236    });
237    if (!record) {
238      throw new ConflictException({
239        code: 'CLINICAL_RECORD_REQUIRED',
240        message: 'Add a screening record before requesting an analysis',
241      });
242    }

```

Figure 4.13: The model router and the fusion step

```
4-ai-intelligence-layer/ai-services/app/router.py lines 43-76

43 def infer(self, request: InferenceRequest) -> InferenceResult:
44     used: list[ModelProvider] = []
45     skipped: list[SkippedModule] = []
46     outputs: dict[str, ModuleOutput] = {}
47
48     for module in self._modules:
49         reason = module.unavailable_reason(request.inputs)
50         if reason:
51             skipped.append(SkippedModule(module=module.module, reason=reason))
52             continue
53         outputs[module.module] = module.run(request, dict(outputs))
54         used.append(module)
55
56     if not used:
57         raise InsufficientInputsError(skipped)
58
59     probability: float | None = None
60     interval: tuple[float, float] | None = None
61     if any(o.probability is not None for o in outputs.values()):
62         fused = self._fusion.run(request, dict(outputs))
63         used.append(self._fusion)
64         probability = fused.probability
65         if probability is not None:
66             interval = (
67                 max(0.0, round(probability - 0.1, 2)),
68                 min(1.0, round(probability + 0.1, 2)),
69             )
70     else:
71         skipped.append(
72             SkippedModule(
73                 module=self._fusion.module,
74                 reason="No module produced a probability to combine",
75             )
76         )
```


Figure 4.14: Fairness check across groups (AUC gap above 0.05 flagged)

```
3-application-logic/backend/src/services/ai/fairness.ts lines 68–107

68 export function fairnessFrom(evaluation: unknown): FairnessReport | null {
69   if (!isRecord(evaluation) || !isRecord(evaluation.byGroup)) return null;
70   const dimensions: FairnessDimension[] = [];
71   for (const [name, raw] of Object.entries(evaluation.byGroup)) {
72     if (!isRecord(raw)) continue;
73     const groups = Object.entries(raw).map([g, figures] => group(g, figures));
74     const aucs = groups.filter(g => g.compared).map(g => g.auc as number);
75     if (aucs.length < 2) {
76       dimensions.push({
77         name,
78         groups,
79         aucGap: null,
80         flagged: false,
81         note: 'Fewer than two groups have enough test cases, so they cannot be compared.',
82       });
83       continue;
84     }
85     // Rounded to the 3 decimals shown, so 0.90 - 0.85 is exactly 0.05 (not
86     // 0.05000000000000004) and is not flagged.
87     const gap =
88       Math.round((Math.max(...aucs) - Math.min(...aucs)) * 1000) / 1000;
89     const flagged = gap > FAIRNESS_AUC_GAP;
90     dimensions.push({
91       name,
92       groups,
93       aucGap: gap,
94       flagged,
95       note: flagged
96         ? `The AUC gap is above ${FAIRNESS_AUC_GAP}: remediation is needed before relying on this model for
all groups.`
97         : null,
98     });
99   }
100   if (dimensions.length === 0) return null;
101   return {
102     threshold: FAIRNESS_AUC_GAP,
103     minCases: FAIRNESS_MIN_CASES,
104     dimensions,
105     flagged: dimensions.some(d => d.flagged),
106   };
107 }
```

Objective 3: the patient education chatbot

Figure 4.15: Safety rules, checked before every answer

```
3-application-logic/backend/src/services/chatbot/chat-safety.ts lines 52-87

52 export function checkQuestion(
53   text: string,
54   audience: ChatAudience,
55 ): SafetyDecision {
56   if (SELF_HARM.test(text)) {
57     return {
58       safety: 'URGENT_CARE',
59       reason: 'self-harm',
60       text: FIXED_TEXT.urgentSelfHarm,
61     };
62   }
63   if (EMERGENCY.test(text)) {
64     return {
65       safety: 'URGENT_CARE',
66       reason: 'emergency',
67       text: FIXED_TEXT.urgentMedical,
68     };
69   }
70   if (MEDICINES.test(text)) {
71     return {
72       safety: 'DECLINED',
73       reason: 'medicines',
74       text: FIXED_TEXT.medicines,
75     };
76   }
77   // Clinicians may ask what a value means in general; patients are never
78   // told what their own results mean (their clinician explains them).
79   if (audience === 'patient' && OWN_RESULTS.test(text)) {
80     return {
81       safety: 'DECLINED',
82       reason: 'own-results',
83       text: FIXED_TEXT.ownResults,
84     };
85   }
86   return { safety: 'OK', reason: 'ok' };
87 }
```

Figure 4.16: The chatbot service: safety, small talk, then the knowledge base

```
3-application-logic/backend/src/services/chatbot/chatbot.service.ts lines 270-327

270  /** Safety check, then the quoted answer, then the output check. */
271  private async answer(
272    text: string,
273    audience: ChatAudience,
274    language: string,
275    earlier: StoredMessage[],
276  ): Promise<{ answer: StoredMessage; reason: string }> {
277    const fixed = (safety: ChatSafety, fixedText: string, reason: string) => ({
278      answer: {
279        id: randomUUID(),
280        role: 'assistant' as const,
281        text: fixedText,
282        at: new Date(),
283        sources: [],
284        safety,
285        mode: 'FIXED' as const,
286      },
287      reason,
288    });
289    const decision = checkQuestion(text, audience);
290    if (decision.safety !== 'OK') {
291      return fixed(decision.safety, decision.text!, decision.reason);
292    }
293    // The number of earlier questions varies repeated replies ("Hi again!",
294    // another joke).
295    const friendly = smallTalk(
296      text,
297      audience,
298      earlier.filter((m) => m.role === 'user').length,
299    );
300    if (friendly) return fixed('OK', friendly, 'small-talk');
301    const noSource =
302      audience === 'patient'
303        ? FIXED_TEXT.noSourcePatient
304        : FIXED_TEXT.noSourceClinician;
305    let result;
306    try {
307      result = await this.ai.chatAnswer({
308        question: text,
309        audience,
310        language,
311        // The last three questions and answers, for follow-ups.
312        history: earlier.slice(-6).map((m) => ({ role: m.role, text: m.text })),
313      });
314    } catch (err) {
315      if (err instanceof AiServiceError && err.status === 409) {
316        throw new ConflictException({
317          code: 'LANGUAGE_NOT_AVAILABLE',
318          message: 'There is no verified content in this language yet.',
319        });
320      }
321      throw new ServiceUnavailableException({
322        code: 'CHAT_UNAVAILABLE',
323        message: 'The assistant is not available right now. Please try later.',
324      });
325    }
326    if (!result.matched) return fixed('NO_SOURCE', noSource, 'no-match');
327    const check = checkAnswer(result.text, result.sources.length);
```

Figure 4.17: Retrieval, step 1: BM25 keyword search of the reviewed knowledge base

```
4-ai-intelligence-layer/ai-services/app/chat/retrieve.py lines 65–100

65 class Retriever:
66     """BM25 over the passages one audience may see."""
67
68     K1 = 1.5
69     B = 0.75
70
71     def __init__(self, kb: KnowledgeBase, audience: Audience, language: str) -> None:
72         # Clinicians may also be given the plain patient information.
73         allowed = {"patient"} if audience == "patient" else {"patient", "clinician"}
74         self.passages = [p for p in kb.passages if p.audience in allowed and p.language == language]
75         # The title and heading count twice: they say what the passage is about.
76         self._docs = [
77             terms(f"{p.title} {p.heading} {p.title} {p.heading} {p.body}") for p in self.passages
78         ]
79         self._freq = [Counter(d) for d in self._docs]
80         n = len(self._docs)
81         self._avg_len = sum(len(d) for d in self._docs) / n if n else 0.0
82         df: Counter[str] = Counter()
83         for d in self._docs:
84             df.update(set(d))
85         self.idf = {t: math.log(1 + (n - c + 0.5) / (c + 0.5)) for t, c in df.items()}
86
87     def search(self, question: str, top_k: int = 5) -> list[Hit]:
88         query = query_terms(question)
89         hits: list[Hit] = []
90         for passage, doc, freq in zip(self.passages, self._docs, self._freq, strict=True):
91             score = 0.0
92             for t in query:
93                 f = freq.get(t, 0)
94                 if f:
95                     norm = self.K1 * (1 - self.B + self.B * len(doc) / self._avg_len)
96                     score += self.idf[t] * f * (self.K1 + 1) / (f + norm)
97             if score > 0:
98                 hits.append(Hit(passage, score))
99         hits.sort(key=lambda h: h.score, reverse=True)
100         return hits[:top_k]
```

Figure 4.18: Retrieval, step 2: the keywords decide, MedCPT orders by meaning (ADR-013)

```

131 def _rank(
132     retriever: Retriever, question: str, text: str, meaning: MeaningScorer | None
133 ) -> tuple[list[Hit], list[Hit]] | None:
134     """The passages in order of relevance, and the ones to quote; None when the
135     knowledge base does not cover the question. `text` is the question, or a
136     short follow-up read together with the previous question."""
137     if meaning is None:
138         hits = retriever.search(text)
139         if not hits or hits[0].score < MIN_SCORE:
140             return None
141         # Whole passages are quoted: each knowledge base section is written to
142         # stand on its own, so a quote never loses the sentence it depends on.
143         quoted = [hits[0]]
144         if (
145             len(hits) > 1
146             and hits[1].score >= SECOND_PASSAGE_RATIO * hits[0].score
147             and len(hits[0].passage.body) + len(hits[1].passage.body) <= MAX_CHARS
148         ):
149             quoted.append(hits[1])
150         return hits, quoted
151
152     # Meaning search: the keywords decide whether the question is covered;
153     # among the passages that match them, the closest in meaning comes first.
154     hits = retriever.search(text, top_k=len(retriever.passages))
155     candidates = [h for h in hits if h.score >= MIN_SCORE]
156     if not candidates:
157         return None
158     closeness = meaning.scores(text, [h.passage for h in candidates])
159     ordered = sorted(zip(candidates, closeness, strict=True), key=lambda pair: -pair[1])
160     (best, best_closeness) = ordered[0]
161     if best_closeness < MEANING_FLOOR:
162         return None
163     # A short question read with the previous one must itself be close to the
164     # passage: "Does it hurt?" after the DRE is, "What is the treatment for
165     # malaria?" after the DRE is not (it is a new topic, and not covered).
166     if text != question and meaning.scores(question, [best.passage])[0] < MEANING_FLOOR:
167         return None
168     quoted = [best]
169     if (
170         len(ordered) > 1
171         and ordered[1][1] >= best_closeness - MEANING_SECOND_GAP
172         and len(best.passage.body) + len(ordered[1][0].passage.body) <= MAX_CHARS
173     ):
174         quoted.append(ordered[1][0])
175     ranked = [h for h, _ in ordered] + [h for h in hits if h.score < MIN_SCORE]
176     return ranked, quoted

```

Figure 4.19: The MedCPT encoder (PubMedBERT, 12 layers) written with numpy

```
4-ai-intelligence-layer/ai-services/app/chat/embeddings.py lines 171-203

171 def _encode(self, ids: Any, mask: Any, types: Any) -> Any:
172     np = self._np
173     batch, length = ids.shape
174     h = (
175         self.w["embeddings.word_embeddings.weight"][ids]
176         + self.w["embeddings.position_embeddings.weight"][:length]
177         + self.w["embeddings.token_type_embeddings.weight"][types]
178     )
179     h = self._norm(h, "embeddings.LayerNorm")
180     size = h.shape[-1] // self.heads
181     # Padding gets no attention.
182     blocked = ((1.0 - mask[:, None, None, :]) * np.finfo(np.float32).min).astype(np.float32)
183
184     def heads(x: Any) -> Any:
185         return x.reshape(batch, length, self.heads, size).transpose(0, 2, 1, 3)
186
187     for i in range(self.layers):
188         p = f"encoder.layer.{i}"
189         q = heads(self._linear(h, f"{p}.attention.self.query"))
190         k = heads(self._linear(h, f"{p}.attention.self.key"))
191         v = heads(self._linear(h, f"{p}.attention.self.value"))
192         scores = q @ k.transpose(0, 1, 3, 2) / math.sqrt(size) + blocked
193         scores = np.exp(scores - scores.max(-1, keepdims=True))
194         probs = scores / scores.sum(-1, keepdims=True)
195         context = (probs @ v).transpose(0, 2, 1, 3).reshape(batch, length, -1)
196         h = self._norm(
197             self._linear(context, f"{p}.attention.output.dense") + h,
198             f"{p}.attention.output.LayerNorm",
199         )
200         inner = self._linear(h, f"{p}.intermediate.dense")
201         inner = 0.5 * inner * (1.0 + _erf(inner / math.sqrt(2.0)))
202         h = self._norm(self._linear(inner, f"{p}.output.dense") + h, f"{p}.output.LayerNorm")
203     return h
```

Figure 4.20: Writing the answer from the passages

```
4-ai-intelligence-layer/ai-services/app/chat/answer.py lines 179-233

179 def answer(
180     kb: KnowledgeBase,
181     request: ChatAnswerRequest,
182     writer: ClaudeWriter | None = None,
183     meaning: MeaningScorer | None = None,
184 ) -> ChatAnswerResult:
185     if request.language not in kb.languages():
186         raise LanguageNotAvailableError(request.language)
187     audience: Audience = request.audience
188     retriever = Retriever(kb, audience, request.language)
189     info = KnowledgeBaseInfo(version=kb.version, reviewStatus=kb.review_status)
190     unmatched = ChatAnswerResult(
191         matched=False, text=None, passages=[], sources=[], knowledgeBase=info
192     )
193     text = _search_text(request)
194     try:
195         ranking = _rank(retriever, request.question, text, meaning)
196     except MeaningBusyError:
197         # Every slot of the meaning search is taken (heavy load): keywords
198         # only, as before ADR-013, so the answer still comes within 2 s.
199         ranking = _rank(retriever, request.question, text, None)
200     if ranking is None:
201         return unmatched
202     hits, quoted = ranking
203
204     if writer is not None:
205         given = hits[:PASSAGES_FOR_CLAUDE]
206         written = writer.write(
207             request.question,
208             audience,
209             [h.passage for h in given],
210             [Turn(role=t.role, text=t.text) for t in request.history],
211         )
212         if written is not None and not written.covered:
213             return unmatched.model_copy(update={"mode": "GENERATED", "model": written.model})
214         if written is not None and written.cited:
215             used = [given[i] for i in written.cited]
216             return ChatAnswerResult(
217                 mode="GENERATED",
218                 matched=True,
219                 text=written.text,
220                 passages=[_passage_out(h) for h in used],
221                 sources=_sources([h.passage for h in used]),
222                 knowledgeBase=_info(kb, [h.passage for h in used]),
223                 model=written.model,
224             )
225         # No usable answer from Claude: quote the passages instead.
226
227     return ChatAnswerResult(
228         matched=True,
229         text=" ".join(h.passage.body for h in quoted),
230         passages=[_passage_out(h) for h in quoted],
231         sources=_sources([h.passage for h in quoted]),
232         knowledgeBase=_info(kb, [h.passage for h in quoted]),
233     )
```

Figure 4.21: The app's chat controller

```
1-presentation-layer/mobile-app/lib/features/chat/application/chat_controller.dart lines 82–117

82 class ChatController extends Notifier<ChatState> {
83   @override
84   ChatState build() => const ChatState();
85
86   Future<void> ask(String text) async {
87     final question = text.trim();
88     if (question.isEmpty || state.sending || state.loading) return;
89     state = state.copyWith(sending: true, clearError: true);
90     final api = ref.read(chatApiProvider);
91     try {
92       final id = state.conversationId ?? await api.start();
93       final (asked, answer) = await api.ask(id, question);
94       state = state.copyWith(
95         conversationId: id,
96         messages: [...state.messages, asked, answer],
97         sending: false,
98       );
99       ref.invalidate(chatHistoryProvider);
100   } catch (e) {
101     state = state.copyWith(sending: false, error: chatErrorText(e));
102   }
103 }
104
105 /// Opens an earlier conversation so it can be read and continued.
106 Future<void> open(String conversationId) async {
107   if (state.sending || conversationId == state.conversationId) return;
108   state = ChatState(conversationId: conversationId, loading: true);
109   try {
110     final messages = await ref.read(chatApiProvider).messages(conversationId);
111     if (state.conversationId != conversationId) return;
112     state = ChatState(conversationId: conversationId, messages: messages);
113   } catch (e) {
114     if (state.conversationId != conversationId) return;
115     state = ChatState(error: chatErrorText(e));
116   }
117 }
```


Figure 4.22: Voice input with the phone's speech service

```
1-presentation-layer/mobile-app/lib/features/chat/application/voice_input.dart lines 74–102

74 Future<void> listen({
75   required void Function(String words) onWords,
76   required void Function(double level) onLevel,
77   required void Function() onDone,
78 }) async {
79   _onDone = onDone;
80   // The answers are in English: use the phone's English locale, or
81   // British English when the phone is set to another language.
82   final system = await _speech.systemLocale();
83   final locale = system != null && system.localeId.startsWith('en')
84     ? system.localeId
85     : 'en_GB';
86   await _speech.listen(
87     onResult: (result) {
88       onWords(result.recognizedWords);
89       if (result.finalResult) _finish();
90     },
91     // Android reports roughly -2 to 10 dB.
92     onSoundLevelChange: (level) => onLevel(((level + 2) / 12).clamp(0, 1)),
93     listenOptions: stt.SpeechListenOptions(
94       listenFor: const Duration(seconds: 60),
95       pauseFor: const Duration(seconds: 3),
96       localeId: locale,
97       partialResults: true,
98       cancelOnError: true,
99       listenMode: stt.ListenMode.dictation,
100   ),
101 );
102 }
```

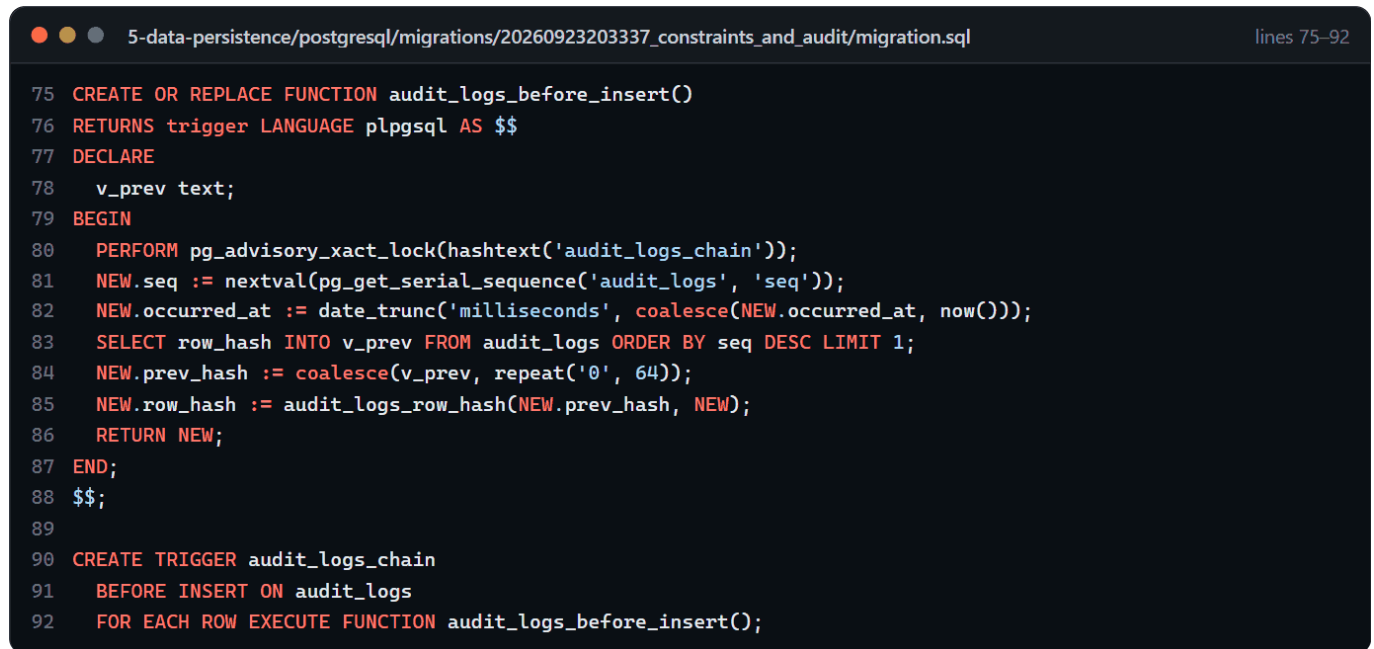
Figure 4.23: Reading aloud for people who cannot read

```
1-presentation-layer/mobile-app/lib/shared/audio/read_aloud.dart lines 110-139

110 class ReadAloudController extends Notifier<ReadAloudState> {
111   int _run = 0;
112   List<String> _parts = const [];
113
114   ReadAloud get _voice => ref.read(readAloudProvider);
115
116   @override
117   ReadAloudState build() => const ReadAloudState();
118
119   Future<void> play(String id, List<String> parts, {int from = 0}) async {
120     final run = ++_run;
121     await _voice.stop();
122     _parts = parts;
123     if (!ref.mounted) return;
124     await _voice.setRate(state.slow ? slowReadingRate : normalReadingRate);
125     for (var i = from; i < parts.length; i++) {
126       // Stopped, or another text started, or the app is closing.
127       if (!ref.mounted || run != _run) return;
128       state = ReadAloudState(
129         id: id,
130         part: i,
131         total: parts.length,
132         slow: state.slow,
133       );
134       await _voice.speak(parts[i]);
135     }
136     if (ref.mounted && run == _run) state = ReadAloudState(slow: state.slow);
137   }
138
139   Future<void> pause() async {
```

Objective 5: ethics, privacy and compliance

Figure 4.24: The audit log's hash chain (database trigger)



```
5-data-persistence/postgresql/migrations/20260923203337_constraints_and_audit/migration.sql lines 75-92

75 CREATE OR REPLACE FUNCTION audit_logs_before_insert()
76 RETURNS trigger LANGUAGE plpgsql AS $$
77 DECLARE
78     v_prev text;
79 BEGIN
80     PERFORM pg_advisory_xact_lock(hashtext('audit_logs_chain'));
81     NEW.seq := nextval(pg_get_serial_sequence('audit_logs', 'seq'));
82     NEW.occurred_at := date_trunc('milliseconds', coalesce(NEW.occurred_at, now()));
83     SELECT row_hash INTO v_prev FROM audit_logs ORDER BY seq DESC LIMIT 1;
84     NEW.prev_hash := coalesce(v_prev, repeat('0', 64));
85     NEW.row_hash := audit_logs_row_hash(NEW.prev_hash, NEW);
86     RETURN NEW;
87 END;
88 $$;
89
90 CREATE TRIGGER audit_logs_chain
91 BEFORE INSERT ON audit_logs
92 FOR EACH ROW EXECUTE FUNCTION audit_logs_before_insert();
```

Figures 4.6 (encryption), 4.10 (de-identification) and 4.14 (fairness) also belong to this objective. **Objective 4** (data collection and model training) has no code yet: no training data or trained model exists in the prototype.

More code pictures in [report/img](#), for example the small-talk replies ([code-chat-small-talk.png](#)), the knowledge base ([code-ai-knowledge-base.png](#)), the assistant's face drawn in code ([code-app-bot-painter.png](#)), the user model ([code-backend-prisma-user.png](#)) and the admin delete dialog ([code-admin-delete-dialog.png](#)).